Building Towards Jolt: Algebraic Foundations of Modern Proof Systems

José Antunes Pedroso de Sousa Franco

Thesis to obtain the Master of Science Degree in

Applied Mathematics and Computation

Supervisor: Prof. Jeroen van de Graaf

IST Supervisor: Prof. Francisco Miguel Alves Campos de Sousa Dionísio

Examination Committee

Chairperson: Prof. Paulo Alexandre Carreira Mateus

Supervisor: Prof. Jeroen van de Graaf

Member of the Committee: Prof. Donald Beaver

October 2025

Declaration / Declaração

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the University of Lisbon.

Declaração

Declaro que o presente documento é um trabalho original da minha autoria e que cumpre todos os requisitos do Código de Conduta e Boas Práticas da Universidade de Lisboa.

Acknowledgments

I would like to express my deepest gratitude to Professor Jeroen van de Graaf for his patience, guidance, and unwavering support throughout this work. His dedication, clarity, and genuine kindness made every step of this process both meaningful and rewarding. I am equally grateful to Professor Paulo Mateus for introducing me to the field of cryptography and zero-knowledge proofs, and for arranging the opportunity that allowed me to work with Professor Jeroen.

Part of the research presented in this dissertation was carried out during my internship at ZKM.IO [ZKM25], and I am grateful for the stimulating research environment in which this work was developed.

I also want to thank my close friends Martim Parreirão, Maria Simões, Simon Hofverberg, and Forsen for their constant support, for keeping me grounded, and for making this period of writing lighter and happier. They turned the hardest parts of this journey into memories I will always look back on fondly, and made this time in my life something I will truly never forget. To all of them, I owe my deepest thanks.

Abstract

This dissertation presents a constructive development of the algebraic foundations that underpin modern zero-knowledge proof systems. It begins with the study of classical interactive proofs and the transition to their algebraic counterparts, introducing Polynomial Commitments, the Schwartz–Zippel lemma, and multilinear extensions as the core tools enabling probabilistic verification. The Fiat–Shamir transform is then examined as the mechanism that removes interaction and yields non-interactive succinct arguments. Building on these foundations, the work explores the Sum-Check and GKR protocols as the central engines of algebraic verification, before extending them to efficient lookup arguments such as Plookup and Logup. Finally, it culminates with Lasso and Jolt, which unify linear verification and modular proof composition into a coherent framework for scalable SNARK construction.

Keywords: Zero-Knowledge Proofs, SNARKs, Polynomial Commitments, Sum-Check, GKR, Logup, Lasso, Jolt

Resumo

Esta dissertação apresenta um desenvolvimento construtivo dos fundamentos algébricos que sustentam os modernos sistemas de prova de conhecimento nulo. O trabalho inicia-se com o estudo das provas interativas clássicas e da sua transição para a forma algébrica, introduzindo Polynomial Commitments, o lema de Schwartz–Zippel e extensões multilineares como ferramentas essenciais para a verificação probabilística. Em seguida, analisa-se a transformação de Fiat–Shamir como o mecanismo que elimina a interação e permite a obtenção de argumentos sucintos e não interativos. Com base nestes fundamentos, são explorados os protocolos Sum-Check e GKR como motores centrais da verificação algébrica, sendo depois estendidos a argumentos de lookup eficientes como Plookup e Logup. Por fim, a dissertação culmina nos sistemas Lasso e Jolt, que unificam a verificação linear e a composição modular de provas num enquadramento coerente para a construção escalável de SNARKs.

Palavras-chave: Zero-Knowledge Proofs, SNARKs, Polynomial Commitments, Sum-Check, GKR, Logup, Lasso, Jolt

Contents

Declaration / Declaração **1**

1 Classical Proof Systems **10**

1.1 A Motivating Example: Quadratic Residues 10

1.1.1 Proving the Statement in a Classical Setting 11

1.1.2 Limitations of Classical Proofs 11

2 Interactive Proof Systems **13**

2.1 The Interactive Proof Model 13

2.2 A Motivating Example: Revisiting the Quadratic Residue Problem 14

2.2.1 The Protocol 15

2.3 Analysis: Completeness and Soundness 16

2.4 From Interaction to Zero-Knowledge 16

3 Zero-Knowledge Proofs **18**

3.1 Motivation and Intuition 18

3.2 The Simulation Paradigm 19

3.2.1 Formal Definition 19

3.2.2 Computational vs. Statistical Indistinguishability 19

3.3 Revisiting the Quadratic Residue Protocol 20

3.3.1 Simulator Construction 20

3.4 Proofs of Knowledge and Extractors 21

3.4.1 Definition 21

3.4.2 Extractor for the Quadratic Residue Protocol 21

4 zk-SNARKs **23**

4.1 From Interactive Proofs to Arguments and SNARKs 23

4.1.1 Non-Interactivity via Fiat–Shamir 23

4.2 Polynomial Commitments and the Schwartz–Zippel Lemma 24

4.2.1 Polynomial Commitments 24

4.2.2 Equality Testing and the Schwartz–Zippel Lemma 26

4.3 Multilinear Extensions and Algebraic Encodings 27

4.4 Arithmetic Circuits and Algebraic Representation 28

4.5 Rank-1 Constraint Systems 28

4.6 From R1CS to Polynomial Form 30

5 The Sum-Check Protocol **31**

5.1 Motivation 31

5.2 Overview of the Protocol 32

5.3 Formal Description of the Protocol 32

5.4 Example 34

5.5 Completeness and Soundness 35

5.6 Completeness and Soundness 35

5.7 Complexity Analysis 36

6 The GKR Protocol **37**

6.1 Goal of the GKR Protocol 37

6.2 Notation and Circuit Representation 38

6.3 Example: Encoding a Small Circuit 39

6.4 Expressing Gate Relations as a Polynomial Identity 39

6.5 The GKR Protocol as an Iterative Application of Sum-Check 40

6.5.1 Initial Step 41

6.5.2 Defining the Target Polynomial for Sum-Check 41

6.5.3 Reduction to a Single Evaluation via Line Interpolation 42

6.6 Final Layer and Termination 43

6.7 Completeness and Soundness 43

6.8 Complexity and Efficiency 44

7 Plookup **45**
7.1 Motivation and Overview . . . 45
7.2 Lookup Relations and Notation . . . 46
7.3 Encoding the Lookup Relation with Polynomials . . . 46
7.4 Correctness of the Encoding . . . 47
7.5 From Algebraic Identity to Protocol . . . 48
7.6 The Plookup Protocol . . . 48
7.6.1 Witness Polynomials . . . 48
7.6.2 Verifier Challenges . . . 48
7.6.3 The Grand Product Polynomial . . . 49
7.6.4 Verifier Checks . . . 49
7.7 Completeness and Soundness . . . 50
7.8 Complexity . . . 51

8 Logup **52**
8.1 Motivation . . . 52
8.2 From Grand Products to Logarithmic Derivatives . . . 52
8.3 The Univariate Logup Protocol . . . 54
8.3.1 Sampling a Random Evaluation Point . . . 54
8.3.2 Expressing the Relation on the Evaluation Domain . . . 54
8.3.3 Completeness and Soundness . . . 55
8.4 Complexity and Efficiency . . . 56
8.5 Comparison with Plookup . . . 56
8.6 Toward the Multivariate Case . . . 57

9 Logup with GKR **58**
9.1 Notation . . . 58
9.2 From Logup to Fractional Sumchecks . . . 59
9.3 GKR for Fractional Sumchecks . . . 60
9.3.1 Projective addition . . . 60
9.3.2 Layer relations . . . 61
9.4 Embedding into Logup . . . 61
9.5 Soundness and Complexity . . . 62

10 Spark: A Building Block of the Lasso Protocol **63**
10.1 Introduction 63
10.2 A Bird's-Eye View of the Spark Protocol 66
10.2.1 The $c = 2$ Case: A Simpler Transformation 67
10.2.2 Evaluation Phase 69
10.2.3 Offline Memory Checking 70
10.2.4 Counter Polynomials and the Final Evaluation Argument 73

11 Lasso: Lookup Arguments from Spark and Surge **75**
11.1 Lasso 77
11.1.1 The road from Spark to Lasso 77
11.1.2 Surge: Extending Spark to Structured Inner Products 80
11.1.3 The Lasso Lookup Argument 82
11.1.4 Cost Analysis and Comparison 83

12 Jolt: Lookup-Based RISC-V Emulation **86**
12.1 Introduction and Motivation 86
12.2 Minimal Machine Model and Problem Statement 87
12.3 Lookup Design 88
12.4 Decomposability Library 89
12.4.1 Core Building Blocks 89
12.4.2 Instruction Composition 90
12.5 Architecture and Composition with Lasso 90
12.5.1 Execution Trace Commitment 90
12.5.2 Lookup Verification with Lasso 91
12.5.3 Virtual Instructions 91
12.5.4 Memory Checking 91
12.6 Costs and Takeaways 92
12.6.1 Per-Step Elements 92
12.6.2 Memory Operations 92
12.6.3 Overall Complexity 93
12.6.4 Conceptual Outlook 93

13 Conclusion **94**

List of Figures

4.1 Arithmetic circuit computing $f(x_1, x_2) = (x_1 + x_2)(x_1 x_2)$. 28

6.1 Example of a layered circuit for the GKR protocol. Each gate has fan-in two, and the layers are indexed from output ($i = 0$) to input ($i = d$). 39

10.1 Example with $c = 3$ and $m = 6$. Each memory block M_i stores all evaluations $\{L_{z_i}(r_i)\}_{z_i}$ of the $\log m$-variate Lagrange basis polynomials. For each nonzero term $z = (z_1, \ldots, z_c)$, the prover looks up $L_{z_i}(r_i)$ from each M_i, multiplies across blocks, multiplies by $g(z)$, and accumulates. Offline memory checking enforces consistency of all reads. 67

Chapter 1

Classical Proof Systems

The concept of proving a statement's validity to a verifier is central in both cryptography and complexity theory. At its core, a **proof system** allows a computationally bounded verifier to become convinced that a statement is true, given evidence provided by a (potentially unbounded) prover.[1] The simplest and most familiar form of such systems is the *classical proof*.

In a classical proof, the prover supplies a concrete *witness* or piece of evidence that enables the verifier to check the claim deterministically. Once the proof is received, the verifier performs a local computation to confirm whether the statement holds. There is no interaction, randomness, or privacy, only direct verification.

1.1 A Motivating Example: Quadratic Residues

To ground this notion, let us examine a simple number-theoretic example. Peggy (the prover) wishes to convince Victor (the verifier) that a number y is a *quadratic residue* modulo n. Formally, Peggy claims that there exists some $x \in \mathbb{Z}_n$ such that

$$y \equiv x^2 \pmod{n}.$$

If such an x exists, we say that y is a square modulo n.

[1] "Computationally bounded" means the verifier runs in polynomial time, while the prover is assumed to have unbounded computational power.

1.1.1 Proving the Statement in a Classical Setting

In the classical model, Peggy simply sends the witness x to Victor. Victor computes $x^2 \bmod n$ and checks whether the result equals y. If it does, Victor accepts, otherwise, he rejects.

This procedure is deterministic, non-interactive, and leaves no room for uncertainty or cheating. For example, suppose

$$n = 11, \quad y = 5.$$

Peggy provides $x = 4$ and claims that 5 is a quadratic residue modulo 11. Victor verifies by computing

$$4^2 = 16 \equiv 5 \pmod{11},$$

confirming the claim.

1.1.2 Limitations of Classical Proofs

While classical proofs are simple and efficient when both parties are honest and computation is cheap, they have two fundamental drawbacks that motivate modern proof systems.

1. Lack of privacy. The witness x must be revealed to the verifier. In many applications, the witness encodes private information such as a secret key, password, or proprietary data. By revealing x, the prover forfeits any privacy guarantees.

2. High verification cost. In other cases, verifying the proof can be as computationally expensive as recomputing the statement from scratch. If the verifier is resource limited for instance, verifying computations outsourced to a more powerful prover, classical proofs offer no advantage.

These limitations motivate the study of richer proof systems that balance efficiency and privacy. In particular, we seek mechanisms where:

- the verifier performs substantially less computation than the prover

- the prover convinces the verifier of a statement's truth without revealing the underlying witness.

These limitations motivated the introduction of randomness into cryptographic protocols, beginning with probabilistic encryption by Goldwasser and Micali [GM84], and later leading to interactive and zero-knowledge proof systems.

Chapter 2

Interactive Proof Systems

Interactive proofs represent one of the most important conceptual shifts in the theory of verification and cryptography. Unlike classical proofs—where the verifier passively checks a fixed witness—interactive proofs introduce a dialogue between the prover and the verifier. Through this exchange, a verifier can be convinced of a statement's validity with high confidence, even without directly learning the witness.

This framework was first formalized independently by Goldwasser, Micali, and Rackoff [GMR89] and by Babai [Bab85], who introduced the notion of interactive proofs as probabilistic verification procedures between a computationally unbounded prover and an efficient verifier. Their works established that interaction and randomness could significantly extend the power of verification beyond classical, deterministic proofs. Subsequent results by Lund, Fortnow, Karloff, and Nisan [Lun+92b], together with Shamir's proof that $\mathsf{IP} = \mathsf{PSPACE}$ [Sha92], demonstrated that the class of languages admitting interactive proofs coincides with PSPACE, a landmark theorem revealing that interaction and randomness together endow verifiers with the full power of polynomial-space computation.

These ideas form the theoretical foundation upon which modern zero-knowledge proofs, succinct arguments, and probabilistic verification systems are built.

2.1 The Interactive Proof Model

An interactive proof is a two-party protocol between:

- a computationally unbounded **prover** $\mathcal{P}$, and

- a probabilistic polynomial-time **verifier** $\mathcal{V}$.

The verifier wishes to be convinced that an input x belongs to some language $L \subseteq \{0,1\}^*$. We denote this interaction as

$$\langle \mathcal{P}, \mathcal{V} \rangle(x).$$

We say that $L \in \mathsf{IP}$ if there exists such a pair $(\mathcal{P}, \mathcal{V})$ satisfying the following two properties.

Completeness. If $x \in L$, then an honest prover convinces the verifier to accept with high probability over the verifier's randomness:

$$\Pr[\langle \mathcal{P}, \mathcal{V} \rangle(x) = 1] \geq 1 - \varepsilon,$$

for negligible ε.

Soundness. If $x \notin L$, then no (even computationally unbounded) cheating prover can convince the verifier to accept except with small probability:

$$\Pr[\langle \mathcal{P}^*, \mathcal{V} \rangle(x) = 1] \leq \varepsilon.$$

Interaction and randomness. The key novelty lies in the exchange of messages and the verifier's ability to generate random challenges. The verifier's randomness makes it impossible for a dishonest prover to anticipate all possible checks. Thus, while the verifier need not recompute the full statement, they can still detect inconsistencies with overwhelming probability.

2.2 A Motivating Example: Revisiting the Quadratic Residue Problem

Let us revisit the quadratic residue example from the previous chapter. Peggy (the prover) wishes to convince Victor (the verifier) that y is a quadratic residue modulo N, i.e., that there exists an x such that

$$y \equiv x^2 \pmod{N}.$$

In the classical setting, Peggy simply revealed x. In the interactive setting, she wants to convince Victor *without revealing* x, while still ensuring that only a valid witness could lead to consistent answers.

2.2.1 The Protocol

1. **Commitment.** Peggy samples a random $r \in \mathbb{Z}_N^*$ and computes

$$s = r^2 \pmod{N}.$$

   She sends s to Victor. Intuitively, s commits to a random quadratic residue without revealing r or x.

2. **Challenge.** Victor samples a random challenge bit $b \in \{0, 1\}$ and sends it to Peggy.

3. **Response.** Peggy responds according to b:

   - If $b = 0$, she sends $z = r$.
   - If $b = 1$, she sends $z = r \cdot x \pmod{N}$.

4. **Verification.** Victor checks:

$$\begin{cases} z^2 \equiv s \pmod{N}, & \text{if } b = 0, \\ z^2 \equiv s \cdot y \pmod{N}, & \text{if } b = 1. \end{cases}$$

   If the check passes, Victor accepts the round.

This simple three-move protocol already illustrates the key intuition of interactive proofs: *the verifier's random challenge forces the prover to demonstrate knowledge consistent with a valid witness, without ever revealing that witness directly.*

2.3 Analysis: Completeness and Soundness

Completeness. If Peggy knows x such that $y \equiv x^2 \pmod{N}$, then she can always answer consistently:

$$z^2 \equiv \begin{cases} r^2 = s, & \text{if } b = 0, \\ (rx)^2 = r^2x^2 = sy, & \text{if } b = 1. \end{cases}$$

Hence, Victor always accepts an honest proof.

Soundness. Suppose Peggy does not know such an x. Then she cannot simultaneously produce z_0, z_1 satisfying:

$$z_0^2 \equiv s \pmod{N} \quad \text{and} \quad z_1^2 \equiv s \cdot y \pmod{N}.$$

Without the square root of y, she can prepare only one of these responses in advance. Therefore, her chance of passing Victor's random challenge is at most $1/2$ per round. By repeating the protocol t times independently, the soundness error drops exponentially:

$$\Pr[\text{cheating prover succeeds}] \leq \left(\frac{1}{2}\right)^t.$$

This repetition converts probabilistic soundness into near certainty—a hallmark of interactive-proof design.

2.4 From Interaction to Zero-Knowledge

The development of interactive proofs revealed a fundamental shift in how verification could be achieved. Rather than requiring the prover to disclose a full witness, interaction and randomness allow the verifier to gain confidence in the truth of a statement through probabilistic consistency checks. This paradigm change inspired a new question at the intersection of cryptography and complexity theory:

> *Can an interactive proof convince the verifier of a statement's truth without revealing anything about* why *it is true?*

This question leads directly to the concept of **zero-knowledge proofs**. Formally introduced by Goldwasser, Micali, and Rackoff [GMR85], zero-knowledge proofs refine

the interactive-proof model to protect the prover's secret information while maintaining completeness and soundness.

Chapter 3

Zero-Knowledge Proofs

In the previous chapter, we saw how interaction and randomness allow a verifier to gain confidence in a statement's truth without necessarily recomputing it from scratch. However, the interactive proof model still leaves one major limitation unresolved: although the prover does not need to reveal the full witness, parts of the interaction may still leak information about it. **Zero-knowledge proofs** (ZKPs) overcome this issue by ensuring that the verifier learns nothing beyond the validity of the statement itself.

3.1 Motivation and Intuition

Returning to our running example, recall the interactive protocol where Peggy convinces Victor that a number y is a quadratic residue modulo N. In that protocol, Peggy's responses depend on the secret square root x, yet Victor learns nothing directly about x except through the consistency checks. Zero-knowledge formalizes this intuition: we want to prove that Victor's entire transcript could have been generated without Peggy's secret knowledge at all.

Informally, if an efficient simulator can reproduce the verifier's view of the protocol without access to the witness, then the interaction reveals no knowledge and is truly *zero-knowledge*.

3.2 The Simulation Paradigm

The simulation paradigm provides the formal definition of zero-knowledge. Originally introduced by Goldwasser, Micali, and Rackoff [GMR89], it compares what the verifier actually observes in a real interaction with what it could have simulated on its own.

3.2.1 Formal Definition

An interactive protocol $(\mathcal{P}, \mathcal{V})$ for a language L is **zero-knowledge** if there exists a probabilistic polynomial-time algorithm Sim (the *simulator*) such that for all $x \in L$,

$$\mathsf{view}_{\mathcal{V}}(\mathcal{P}, \mathcal{V})[x] \approx_c \mathsf{Sim}(x),$$

where the verifier's view includes its random coins and all messages exchanged during the protocol. The notation $\approx_c$ denotes *computational indistinguishability*, meaning that no efficient distinguisher can tell the two distributions apart with non-negligible advantage.

Intuition. If the verifier's entire experience of the protocol can be generated without the prover's secret, then the interaction itself conveys no new information. Whatever the verifier learns could have been simulated independently, hence "zero-knowledge."

3.2.2 Computational vs. Statistical Indistinguishability

Indistinguishability comes in varying strengths, depending on how powerful the adversary (or distinguisher) is assumed to be.

Computational indistinguishability. Two distributions are computationally indistinguishable if no efficient (PPT) distinguisher can tell them apart except with negligible probability. This is the standard notion used in modern cryptography and relies on computational hardness assumptions.

Statistical indistinguishability. Two distributions are statistically indistinguishable if they are close in total variation distance, even for an unbounded distinguisher. This is a stronger, information-theoretic guarantee: statistical zero-knowledge protocols remain private even against infinitely powerful verifiers.

In short,

$$\text{Perfect ZK} \;\Rightarrow\; \text{Statistical ZK} \;\Rightarrow\; \text{Computational ZK}.$$

In modern cryptographic applications, the assumption of **computational indistinguishability** is the prevailing one. Security relies on standard hardness assumptions such as the intractability of integer factorization, discrete logarithms, or lattice problems, ensuring that no efficient adversary can distinguish the simulated view from a real execution.

3.3 Revisiting the Quadratic Residue Protocol

Let us now revisit the quadratic residue protocol from Chapter 2, this time focusing on the zero-knowledge property. Recall that Peggy wishes to convince Victor that there exists an x such that

$$y \equiv x^2 \pmod{N}.$$

The three-move protocol commitment $s = r^2$, random challenge b, and response $z = r$ or $r \cdot x$ already satisfies completeness and soundness. We now show that it also satisfies **perfect zero-knowledge** (against an honest verifier).

3.3.1 Simulator Construction

To argue zero-knowledge, we construct an explicit simulator Sim that reproduces Victor's view without knowing x. The simulator operates as follows:

1. Choose a random bit $b \in \{0, 1\}$.
2. Sample $z \in \mathbb{Z}_N^*$ uniformly at random.
3. Compute s so that Victor's verification would pass:

$$s = \begin{cases} z^2 \pmod{N}, & \text{if } b = 0, \\ z^2 \cdot y^{-1} \pmod{N}, & \text{if } b = 1. \end{cases}$$

The resulting transcript (s, b, z) is identically distributed to that of a genuine protocol execution between Peggy and Victor. Thus, even an unbounded verifier cannot distinguish

simulated from real transcripts: the protocol is **perfect zero-knowledge**.

3.4 Proofs of Knowledge and Extractors

Zero-knowledge alone ensures privacy, but not necessarily that the prover *knows* the claimed witness. For that, we require an additional property: **knowledge soundness**. A protocol that is both sound and extractable is called a **proof of knowledge**.

3.4.1 Definition

An interactive protocol $(\mathcal{P}, \mathcal{V})$ for a relation $R(x, w)$ is a proof of knowledge if there exists an efficient extractor Ext such that, whenever a prover $\mathcal{P}^*$ convinces the verifier with non-negligible probability, $\mathsf{Ext}^{\mathcal{P}^*}(x)$ outputs a valid witness w satisfying $(x, w) \in R$.

3.4.2 Extractor for the Quadratic Residue Protocol

The extractor operates by "rewinding" the same initial commitment and issuing both challenges to the prover:

1. Receive the prover's commitment $s = r^2 \pmod{N}$.
2. Challenge once with $b = 0$, obtaining response $z_0 = r$.
3. Rewind to the same commitment and challenge again with $b = 1$, obtaining $z_1 = r \cdot x \pmod{N}$.
4. Compute the hidden witness as

$$x \equiv z_1 \cdot z_0^{-1} \pmod{N}.$$

If the prover can successfully answer both challenges, the extractor recovers x efficiently, thus proving that any convincing prover must indeed *know* the square root of y.

The quadratic residue example thus encapsulates the core properties of zero-knowledge proofs:

- **Completeness:** an honest prover always convinces the verifier.

- **Soundness:** a dishonest prover cannot cheat except with negligible probability.
- **Zero-Knowledge:** a verifier learns nothing beyond the truth of the statement.

This combination of verifiability and privacy—together with the extraction property—defines the modern notion of a *zero-knowledge proof of knowledge*. These principles form the foundation of advanced constructions such as succinct non-interactive proofs (SNARKs), which we study next.

Chapter 4

zk-SNARKs

A **Zero-Knowledge Succinct Non-Interactive Argument of Knowledge** (zk-SNARK) is a cryptographic proof system that enables a prover to convince a verifier of a statement's truth while revealing nothing beyond that fact, and doing so with extremely efficient verification.

We now build up the main ingredients that make zk-SNARKs possible.

4.1 From Interactive Proofs to Arguments and SNARKs

In a traditional *interactive proof*, completeness and soundness are guaranteed even against computationally unbounded provers. However, to obtain succinct and efficiently verifiable proofs, we typically relax soundness to hold only against efficient (PPT) provers. This leads to the notion of a **computationally sound argument** [Tha22].

4.1.1 Non-Interactivity via Fiat–Shamir

Most interactive proof protocols—including the zero-knowledge protocol for quadratic residues discussed earlier, require the verifier to generate random challenges. In their seminal work, Fiat and Shamir [FS87] proposed a heuristic to remove this interaction by replacing the verifier's randomness with a cryptographic hash. The challenge b is computed deterministically as

$$b = H(\text{commitment}, \text{statement}),$$

where H is modeled as a random oracle.

The Fiat–Shamir transform thus provides the standard bridge from interactive arguments of knowledge to the non-interactive zk-SNARKs used in practice.

4.2 Polynomial Commitments and the Schwartz–Zippel Lemma

In succinct proof systems, the verifier must be convinced that certain algebraic relations hold among polynomials that encode a computation. Revealing the full coefficients of these polynomials would be inefficient and often leak private information. A *polynomial commitment scheme* allows the prover to commit to a polynomial $f(X)$ once and later open the commitment at selected points, while convincing the verifier of correctness with minimal communication.

4.2.1 Polynomial Commitments

Formally, a polynomial commitment scheme provides three algorithms:

$$(\mathsf{Setup}, \mathsf{Commit}, \mathsf{VerifyEval})$$

such that:

- The prover computes a commitment $\mathsf{Com}(f)$ to a polynomial $f(X)$ under public parameters.
- For a chosen evaluation point $z \in \mathbb{F}$, the prover produces an opening proof that the committed polynomial satisfies $f(z) = v$.
- The verifier, using $\mathsf{Com}(f)$ and the proof, checks this relation without learning anything else about f.

The commitment must be **binding**, ensuring the prover cannot later open it to a different polynomial, and **succinctly verifiable**, meaning the verifier's work is constant or logarithmic in the polynomial's degree.

Example: the KZG Commitment. The KZG construction provides a succinct and binding commitment to a polynomial by encoding its evaluation at a hidden point. Let a trusted setup generate a structured reference string

$$(g, g^{\tau}, g^{\tau^2}, \ldots, g^{\tau^d}),$$

where g is a generator of a cryptographic group and $\tau \in \mathbb{F}$ is a secret value known to no party. Given a polynomial $f(X) = \sum_{i=0}^{d} a_i X^i$, the commitment is defined as

$$\mathsf{Com}(f) = g^{f(\tau)} = \prod_{i=0}^{d} (g^{\tau^i})^{a_i}.$$

This commitment is homomorphic and hides the coefficients of f while binding the prover to a single polynomial consistent with the public parameters.

To prove that $f(z) = v$ for some public evaluation point $z \in \mathbb{F}$, the prover constructs the quotient polynomial

$$q(X) = \frac{f(X) - v}{X - z}.$$

If the claimed evaluation is correct, this quotient is well-defined and satisfies $f(X) - v = q(X)(X - z)$. The prover then sends the commitment $\mathsf{Com}(q) = g^{q(\tau)}$, and the verifier must be convinced that

$$g^{f(\tau)-v} = g^{q(\tau)(\tau-z)}.$$

This equality expresses precisely the divisibility condition that guarantees $f(z) = v$. However, verifying it directly would require knowledge of τ, which is secret. The challenge, therefore, is to check the relation in the exponent without revealing or reconstructing τ.

This is achieved through a bilinear pairing $e : G_1 \times G_2 \to G_T$ satisfying $e(g^a, h^b) = e(g, h)^{ab}$. The verifier tests the single equation

$$e(\mathsf{Com}(f)/g^v, g) = e(\mathsf{Com}(q), g^{\tau-z}),$$

which holds if and only if $f(\tau) - v = q(\tau)(\tau - z)$. The pairing allows multiplicative reasoning about exponents while keeping τ hidden, thereby preserving soundness and succinctness.

4.2.2 Equality Testing and the Schwartz–Zippel Lemma

At the heart of polynomial verification lies the question of equality testing. Given two polynomials $f, g \in \mathbb{F}[X_1, \ldots, X_n]$, a verifier needs to confirm that $f = g$ without evaluating them exhaustively. The Schwartz–Zippel lemma provides the probabilistic guarantee that underpins this approach.

Lemma 4.1 (Schwartz–Zippel [Sch80; Zip79])**.** *Let $f(X_1, \ldots, X_n)$ be a nonzero polynomial of total degree d over a finite field $\mathbb{F}$. If a random point $r = (r_1, \ldots, r_n) \in \mathbb{F}^n$ is sampled uniformly, then*

$$\Pr[f(r) = 0] \leq \frac{d}{|\mathbb{F}|}.$$

Proof. The result follows by induction on the number of variables n. For the base case $n = 1$, let $f(X_1)$ be a nonzero univariate polynomial of degree d. Such a polynomial has at most d roots in $\mathbb{F}$, so if r_1 is chosen uniformly from $\mathbb{F}$,

$$\Pr[f(r_1) = 0] \leq \frac{d}{|\mathbb{F}|}.$$

Assume the claim holds for all polynomials in $n - 1$ variables, and let $f(X_1, \ldots, X_n)$ be a nonzero polynomial of total degree d. Write it as a univariate polynomial in X_n,

$$f(X_1, \ldots, X_n) = \sum_{i=0}^{k} f_i(X_1, \ldots, X_{n-1}) X_n^i,$$

where f_k is not identically zero and $k \leq d$. For a fixed choice of $(r_1, \ldots, r_{n-1})$, either all coefficients $f_i(r_1, \ldots, r_{n-1})$ vanish—an event occurring with probability at most $\deg(f_k)/|\mathbb{F}|$ by the inductive hypothesis—or $f(r_1, \ldots, r_{n-1}, X_n)$ is a nonzero univariate polynomial of degree at most k, which has at most k roots.

Hence,

$$\Pr[f(r_1, \ldots, r_n) = 0] \leq \Pr[f_k(r_1, \ldots, r_{n-1}) = 0] + \frac{k}{|\mathbb{F}|} \leq \frac{d}{|\mathbb{F}|}.$$

□

In particular, if two distinct polynomials f and g differ on even one coefficient, then $f(r) \neq g(r)$ for a random r except with probability $d/|\mathbb{F}|$. Thus a single random evalua-

tion suffices to verify equality with high confidence. This principle is fundamental to all algebraic proof systems, including the sum-check and GKR protocols [Tha22].

4.3 Multilinear Extensions and Algebraic Encodings

Let $f : \{0,1\}^\ell \to \mathbb{F}$ be a Boolean function. Its **multilinear extension** $\widetilde{f} : \mathbb{F}^\ell \to \mathbb{F}$ is the unique polynomial that has degree at most one in each variable and agrees with f on all Boolean points:

$$\widetilde{f}(y) = \sum_{x \in \{0,1\}^\ell} f(x) \prod_{i=1}^{\ell} \big(x_i y_i + (1 - x_i)(1 - y_i)\big).$$

Each product term acts as an indicator polynomial that evaluates to one when $y = x$ and to zero otherwise.

A polynomial is *multilinear* if every variable appears with degree at most one. This restriction ensures that no term involves powers such as X_i^2 or $X_i X_j^2$; each variable contributes linearly and independently. Over the Boolean hypercube, this property guarantees that the polynomial behaves predictably under restriction to any subset of coordinates. In particular, fixing one variable of a multilinear polynomial yields another multilinear polynomial in the remaining variables. This recursive structure makes multilinear polynomials well-suited to the layered reductions used in interactive proofs.

The space of multilinear polynomials in ℓ variables has dimension 2^ℓ, equal to the number of Boolean assignments in $\{0,1\}^\ell$. Each such polynomial can be represented in the *Lagrange basis*, indexed by these assignments, or equivalently as a tensor product of one-dimensional spaces. This tensor view makes explicit how evaluations and interpolations can be decomposed efficiently along coordinates, a property that becomes central in protocols such as sumcheck and GKR.

Multilinear extensions allow discrete computations to be expressed as algebraic objects defined over the entire field $\mathbb{F}$. Once represented in this way, circuit constraints or table lookups can be checked through polynomial equalities (using the Schwartz–Zippel Lemma) rather than combinatorial enumeration. As observed in [Lam23], this algebraic formulation provides a compact and structured setting for reasoning about computations over large domains while maintaining low degree and efficient evaluation.

These properties make multilinear extensions a fundamental component of modern proof systems.

4.4 Arithmetic Circuits and Algebraic Representation

To express general computation in algebraic form, zk-SNARKs represent the verifier's predicate as an **arithmetic circuit** over a finite field $\mathbb{F}$. This abstraction is fundamental because any efficiently computable function can be compiled into such a circuit. A proof system can then reason about the correctness of the computation through algebraic identities over $\mathbb{F}$.

Formally, an arithmetic circuit is a directed acyclic graph composed of addition and multiplication gates whose inputs and outputs are field elements. Each valid execution corresponds to an assignment of wire values that satisfies every gate constraint. Thus, verifying that a computation was carried out correctly reduces to checking that a collection of polynomial equalities hold.

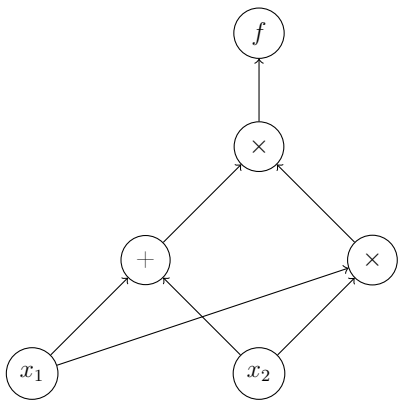

Figure 4.1: Arithmetic circuit computing $f(x_1, x_2) = (x_1 + x_2)(x_1 x_2)$.

Every language in NP admits such a representation. A verifier's predicate $V(x, w)$ that checks whether a public statement x and a private witness w satisfy a relation $V(x, w) = 1$ can be compiled into a circuit. The correctness of the computation is then expressed as the existence of an assignment of wire values that satisfies all gate equations. This assignment, denoted by $\mathbf{z}$, plays the role of the extended witness containing inputs, intermediate values, and outputs. Proving that $V(x, w) = 1$ becomes equivalent to proving the existence of a vector $\mathbf{z}$ consistent with all gate constraints.

4.5 Rank-1 Constraint Systems

A **Rank-1 Constraint System (R1CS)** formalizes the constraints imposed by an arithmetic circuit as a set of quadratic equations over $\mathbb{F}$, a representation that arose in the

study of quadratic arithmetic programs for succinct non-interactive arguments [Gen+13].

Each constraint asserts that the product of two linear combinations of wire values equals another linear combination. This provides a uniform algebraic framework to represent both addition and multiplication gates.

Let

$$\mathbf{z} = (1, w_1, w_2, \ldots, w_n)$$

denote the vector containing all wire values, including a leading constant 1. Each gate is described by three coefficient vectors

$$\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{F}^{n+1},$$

and contributes a constraint of the form

$$(\mathbf{a} \cdot \mathbf{z})(\mathbf{b} \cdot \mathbf{z}) = (\mathbf{c} \cdot \mathbf{z}).$$

Here, $\mathbf{a} \cdot \mathbf{z}$ and $\mathbf{b} \cdot \mathbf{z}$ are linear combinations corresponding to the inputs of a gate, while $\mathbf{c} \cdot \mathbf{z}$ corresponds to its output. If every constraint in the system is satisfied, then $\mathbf{z}$ represents a valid execution of the circuit.

Example. For the circuit computing $f(x_1, x_2) = (x_1 + x_2)(x_1 x_2)$, we define intermediate variables

$$w_3 = x_1 + x_2,$$

$$w_4 = x_1 x_2,$$

$$w_5 = w_3 w_4.$$

With wire vector $\mathbf{z} = (1, x_1, x_2, w_3, w_4, w_5)$, the corresponding constraints are:

$$(x_1 + x_2) \cdot 1 = w_3,$$

$$x_1 \cdot x_2 = w_4,$$

$$w_3 \cdot w_4 = w_5.$$

Each equation has the canonical R1CS form $(\mathbf{a}_i \cdot \mathbf{z})(\mathbf{b}_i \cdot \mathbf{z}) = (\mathbf{c}_i \cdot \mathbf{z})$, defining one constraint for each gate.

4.6 From R1CS to Polynomial Form

The R1CS representation can be expressed as a single polynomial identity. Let the constraints be indexed by distinct elements of a finite domain

$$H = \{t_1, t_2, \ldots, t_m\} \subset \mathbb{F}.$$

Define polynomials $A(X)$, $B(X)$, and $C(X)$ by interpolation such that for each $t_i \in H$,

$$A(t_i) = \mathbf{a}_i \cdot \mathbf{z}, \qquad B(t_i) = \mathbf{b}_i \cdot \mathbf{z}, \qquad C(t_i) = \mathbf{c}_i \cdot \mathbf{z}.$$

If $\mathbf{z}$ satisfies all constraints, then

$$A(X)B(X) - C(X)$$

vanishes on H. Let $Z_H(X) = \prod_{t \in H}(X - t)$ denote the vanishing polynomial of H. The R1CS condition is therefore equivalent to the divisibility relation

$$A(X)B(X) - C(X) = h(X) \cdot Z_H(X)$$

for some quotient polynomial $h(X)$.

This formulation aggregates all constraints into a single polynomial identity, eliminating the need to verify each gate separately. A verifier must only check that the identity holds at a random evaluation point, which by the Schwartz–Zippel lemma ensures correctness with overwhelming probability. This algebraic reduction underlies many practical zk-SNARK constructions, most notably the Groth16 protocol [Gro16], and is explained in detail from a proof-theoretic perspective in [Tha22].

Chapter 5

The Sum-Check Protocol

The **sum-check protocol** is one of the central algebraic proof systems in the modern theory of interactive proofs and succinct arguments, first introduced by Lund, Fortnow, Karloff, and Nisan [Lun+92a]. It provides a way for a prover to convince a verifier of the value of an exponentially large sum of polynomial evaluations, using only a logarithmic number of rounds and a small amount of communication.

In the previous chapters, we have progressively developed the algebraic ingredients that allow computational statements to be verified probabilistically and succinctly: multilinear extensions transformed discrete Boolean computations into low-degree polynomials over a field; the Schwartz–Zippel lemma provided a way to test equality between polynomials at random points; and polynomial commitments gave a cryptographic interface for hiding and verifying these evaluations. The sum-check protocol combines these ideas into a powerful interactive procedure that underlies many zero-knowledge proof systems, including the GKR protocol and modern zk-SNARK constructions.

5.1 Motivation

Let $g : \mathbb{F}^v \to \mathbb{F}$ be a v-variate polynomial over a finite field $\mathbb{F}$, with degree at most d in each variable. We are interested in computing the sum of g over the Boolean hypercube $\{0,1\}^v$:

$$H := \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \cdots \sum_{b_v \in \{0,1\}} g(b_1, \ldots, b_v). \tag{5.1}$$

As one can see, the sum in (5.1) involves 2^v terms, which is prohibitively large for

the verifier to compute directly. The *sum-check protocol* allows a prover P to convince a verifier V that H equals the correct value, using only $O(v)$ communication and $O(v)$ verifier work plus a single evaluation of g.

Although the task of summing a polynomial over the Boolean cube might initially appear artificial, it will later emerge naturally when verifying layered arithmetic circuits via the GKR protocol. We defer a detailed discussion of this protocol to Chapter 6.

5.2 Overview of the Protocol

The sum-check protocol is an interactive proof system for the following relation:

$$R = \big\{(g, H) : H = \sum_{b \in \{0,1\}^v} g(b)\big\},$$

where g is given as an oracle and $H \in \mathbb{F}$ is the claimed sum.

The key idea is to recursively expand the sum dimension by dimension:

$$H = \sum_{b_1 \in \{0,1\}} \sum_{b_2,\ldots,b_v \in \{0,1\}} g(b_1, \ldots, b_v) = \sum_{b_1 \in \{0,1\}} g_1(b_1),$$

where

$$g_1(X_1) := \sum_{(x_2,\ldots,x_v) \in \{0,1\}^{v-1}} g(X_1, x_2, \ldots, x_v).$$

If the prover can convince the verifier that $g_1(X_1)$ is indeed the correct polynomial and that $H = g_1(0) + g_1(1)$, then it remains to verify correctness of g_1 which itself can be checked by another application of the same idea, on a smaller-dimensional polynomial.

5.3 Formal Description of the Protocol

Definition 5.1 (Sum-Check Protocol)**.** Let $g : \mathbb{F}^v \to \mathbb{F}$ be a polynomial of individual degree at most d in each variable, and let $H \in \mathbb{F}$ be the claimed sum (5.1). The sum-check protocol proceeds as follows.

1. **Initialization.** The prover P sends to V a value C_1 claimed to equal H.

2. **First round.** The prover sends a univariate polynomial

$$g_1(X_1) = \sum_{(x_2,\ldots,x_v)\in\{0,1\}^{v-1}} g(X_1, x_2, \ldots, x_v).$$

The verifier checks:

$$C_1 = g_1(0) + g_1(1),$$

and verifies that $\deg(g_1) \leq \deg_1(g)$. If either check fails, V rejects.

3. **Subsequent rounds.** For each round $j = 2, \ldots, v$:

   (a) V chooses a random field element $r_{j-1} \in \mathbb{F}$ and sends it to P.

   (b) P sends a univariate polynomial

$$g_j(X_j) = \sum_{(x_{j+1},\ldots,x_v)\in\{0,1\}^{v-j}} g(r_1, \ldots, r_{j-1}, X_j, x_{j+1}, \ldots, x_v).$$

   (c) V checks that $\deg(g_j) \leq \deg_j(g)$ and that

$$g_{j-1}(r_{j-1}) = g_j(0) + g_j(1).$$

   If any check fails, V rejects.

4. **Final round.** In the final round ($j = v$), after P sends $g_v(X_v)$, the verifier samples a random $r_v \in \mathbb{F}$ and performs a single oracle query to g at the point $(r_1, \ldots, r_v)$. The verifier then checks:

$$g_v(r_v) = g(r_1, \ldots, r_v).$$

If this check passes and no previous check has failed, V accepts.

Intuitively, each round reduces the dimensionality of the claimed sum by one. After the first round, verifying the correctness of H reduces to verifying correctness of g_1. Each subsequent round replaces a summation over $\{0,1\}^{v-j}$ with a single random evaluation, and after v rounds, the verifier only needs to evaluate g once—at a random point.

5.4 Example

To illustrate the mechanics of the protocol, consider

$$g(X_1, X_2, X_3) = 2X_1^3 + X_1X_3 + X_2X_3$$

over a finite field $\mathbb{F}$. The goal is to compute

$$H = \sum_{b_1,b_2,b_3\in\{0,1\}} g(b_1, b_2, b_3) = 12.$$

Round 1. The prover first sends

$$g_1(X_1) = \sum_{x_2,x_3\in\{0,1\}} g(X_1, x_2, x_3) = (2X_1^3)+(2X_1^3+X_1)+(2X_1^3)+(2X_1^3+X_1+1) = 8X_1^3+2X_1+1.$$

The verifier checks $g_1(0) + g_1(1) = 12$, confirming that the claimed sum is consistent so far, and sends a random challenge $r_1 = 2$.

Round 2. The prover computes

$$g_2(X_2) = \sum_{x_3\in\{0,1\}} g(2, X_2, x_3) = 16 + (16 + 2 + X_2) = 34 + X_2.$$

The verifier checks $g_2(0) + g_2(1) = g_1(r_1)$, which indeed holds since both sides equal 69, and then sends $r_2 = 3$.

Round 3. Finally, the prover sends

$$g_3(X_3) = g(2, 3, X_3) = 16 + 5X_3.$$

The verifier checks $g_3(0) + g_3(1) = g_2(r_2)$, which is $37 = 37$, and samples $r_3 = 6$. The verifier evaluates $g(2, 3, 6) = 46$, confirms $g_3(r_3) = 46$, and accepts.

5.5 Completeness and Soundness

5.6 Completeness and Soundness

The correctness and complexity analyses presented in this section follow the standard treatment of the sum-check protocol as introduced by Lund, Fortnow, Karloff, and Nisan [Lun+92a], and are included here for completeness.

We now show that the sum-check protocol is an interactive proof system for the relation

$$L = \big\{(g, H) : H = \sum_{b \in \{0,1\}^v} g(b)\big\},$$

with perfect completeness and small soundness error.

Theorem 5.2 (Soundness and Completeness of the Sum-Check Protocol)**.** *Let $g : \mathbb{F}^v \to \mathbb{F}$ be a polynomial of individual degree at most d in each variable. For any claimed value $H \in \mathbb{F}$, the sum-check protocol satisfies:*

- ***Perfect completeness:*** *an honest prover always causes the verifier to accept.*
- ***Soundness error:*** *any dishonest prover can convince the verifier of an incorrect claim with probability at most $\frac{vd}{|\mathbb{F}|}$.*

Proof. **Completeness.** In round j, an honest prover sends

$$g_j(X_j) = \sum_{(x_{j+1},\ldots,x_v) \in \{0,1\}^{v-j}} g(r_1, \ldots, r_{j-1}, X_j, x_{j+1}, \ldots, x_v),$$

which is precisely the polynomial that encodes the partial sum of g over the remaining variables. Thus, every verifier check

$$g_{j-1}(r_{j-1}) = g_j(0) + g_j(1), \qquad g_v(r_v) = g(r_1, \ldots, r_v)$$

holds identically, and the verifier always accepts. Hence completeness is perfect.

Soundness. Suppose the prover's claim H is incorrect. Then for some round i, the polynomial $g_i(X_i)$ sent by the prover differs from the honest polynomial $s_i(X_i)$ defined above. Let B_i be the event that the verifier accepts all checks up to round i despite this discrepancy.

In the first round, if $g_1 \neq s_1$, both are degree-d polynomials, and by the Schwartz–Zippel lemma

$$\Pr[g_1(r_1) = s_1(r_1)] \leq \tfrac{d}{|\mathbb{F}|}.$$

Assuming inductively that $\Pr[B_{i-1}] \leq (i-1)\frac{d}{|\mathbb{F}|}$, another incorrect polynomial in round i can pass its check only if $g_i(r_i) = s_i(r_i)$, which occurs with probability at most $\frac{d}{|\mathbb{F}|}$. By a union bound over the v rounds,

$$\Pr[\text{verifier accepts a false claim}] \leq \tfrac{vd}{|\mathbb{F}|}.$$

□

5.7 Complexity Analysis

The protocol runs for exactly v rounds, one per variable of g.

Communication. In each round the prover sends one univariate polynomial of degree at most $\deg_j(g)$, requiring $\deg_j(g) + 1$ field elements, and the verifier replies with one random challenge. The total communication is

$$O\Big(v + \sum_{j=1}^{v} \deg_j(g)\Big)$$

field elements.

Prover time. To form $g_j(X_j)$, the prover sums g over all 2^{v-j} Boolean assignments to the remaining variables, for each of the $(\deg_j(g) + 1)$ evaluation points. The total work is

$$O\left(\sum_{j=1}^{v} (\deg_j(g) + 1)\, 2^{v-j}\right),$$

only a constant factor above computing the full sum directly.

Verifier time. The verifier performs $O(v)$ field operations and a single oracle query to g at $(r_1, \ldots, r_v)$, an exponential improvement over recomputing the entire sum.

Chapter 6

The GKR Protocol

The **Goldwasser–Kalai–Rothblum (GKR) protocol** is a landmark result in the theory of interactive proofs, introduced by Goldwasser, Kalai, and Rothblum [GKR15]. It provides an efficient method for verifying the correctness of computations represented by layered arithmetic circuits. The verifier's work grows only logarithmically with the size of the circuit, while the prover's work is nearly linear in the size of the computation. Together with the **sum-check protocol**, it forms one of the principal algebraic backbones for modern SNARK constructions.

6.1 Goal of the GKR Protocol

Consider an arithmetic circuit C with fan-in two and depth d, where each layer i contains 2^{k_i} gates. The output of the circuit lies in layer 0, and the inputs lie in layer d. The prover P wishes to convince the verifier V that the circuit, when evaluated on some input vector x, produces a claimed output y.

Formally, the goal of the GKR protocol is to verify the relation

$$y = C(x),$$

with communication and verification cost polylogarithmic in the circuit size.

6.2 Notation and Circuit Representation

We index layers from top to bottom:

$$i = 0, 1, \ldots, d,$$

where layer 0 contains the output gates, and layer d the input gates. Each gate in layer i is identified by a binary string of length k_i, that is, by an element of $\{0,1\}^{k_i}$.

The W polynomials. For each layer i, define

$$W_i : \{0,1\}^{k_i} \to \mathbb{F}$$

so that $W_i(a)$ is the value of gate a in layer i when the circuit is evaluated on input x. The multilinear extension of W_i, denoted $\widetilde{W_i} : \mathbb{F}^{k_i} \to \mathbb{F}$, is the unique multilinear polynomial agreeing with W_i on Boolean inputs.

In-neighbor functions. Let $\mathrm{in}_{1,i}$ and $\mathrm{in}_{2,i}$ denote the wiring functions that specify the parents of each gate:

$$\mathrm{in}_{1,i}, \mathrm{in}_{2,i} : \{0,1\}^{k_i} \to \{0,1\}^{k_{i+1}}.$$

If a is a gate at layer i, then $\mathrm{in}_{1,i}(a)$ and $\mathrm{in}_{2,i}(a)$ denote the two gates at layer $i+1$ feeding into a.

Wiring predicates. The circuit's structure can be captured by two Boolean functions:

$$\mathrm{add}_i,\ \mathrm{mult}_i : \{0,1\}^{k_i+2k_{i+1}} \to \{0,1\},$$

where $\mathrm{add}_i(a,b,c) = 1$ if gate a at layer i is an addition gate with inputs (b,c), and $\mathrm{mult}_i(a,b,c) = 1$ if it is a multiplication gate with inputs (b,c). Their multilinear extensions $\widetilde{\mathrm{add}}_i$ and $\widetilde{\mathrm{mult}}_i$ will be central to the protocol.

6.3 Example: Encoding a Small Circuit

Figure 6.1 shows a two-layer circuit computing two multiplication gates followed by one final multiplication. The wiring predicates for each layer can be expressed directly in terms of Boolean triples (a, b, c), where a indexes the gate in layer i, and (b, c) its parents in layer $i + 1$.

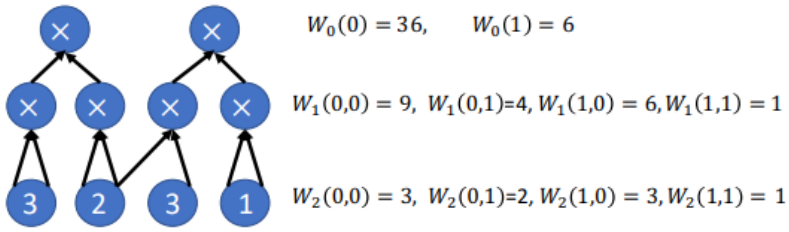

Figure 6.1: Example of a layered circuit for the GKR protocol. Each gate has fan-in two, and the layers are indexed from output $(i = 0)$ to input $(i = d)$.

In this example, suppose layer 0 has two multiplication gates and no addition gates. Then add_0 is the constant-zero function, while $\mathrm{mult}_0(a, b, c) = 1$ only for those triples (a, b, c) consistent with the wiring shown. The same structure applies recursively to lower layers, fully determining the circuit's topology.

Remark. The GKR protocol never needs to represent the circuit explicitly as a graph. Instead, it manipulates these multilinear encodings of gate values and wiring relations, allowing the entire verification to occur in the algebraic domain.

6.4 Expressing Gate Relations as a Polynomial Identity

To verify that the values of layer i are consistent with layer $i+1$, the GKR protocol relies on the following algebraic identity, which expresses each gate's value as a function of its parents.

Lemma 6.1 (Layer relation)**.** *For every layer $i \in \{0, \ldots, d-1\}$ and every $z \in \mathbb{F}^{k_i}$,*

$$\widetilde{W}_i(z) = \sum_{b,c \in \{0,1\}^{k_{i+1}}} \Big(\widetilde{\mathrm{add}}_i(z, b, c) \big[\widetilde{W}_{i+1}(b) + \widetilde{W}_{i+1}(c) \big] + \widetilde{\mathrm{mult}}_i(z, b, c) \big[\widetilde{W}_{i+1}(b) \widetilde{W}_{i+1}(c) \big] \Big).$$

Proof. Both sides are multilinear in z. By the uniqueness of multilinear extensions, it suffices to verify equality on Boolean inputs $z = a \in \{0,1\}^{k_i}$. Fix a gate a at layer i. If a is an addition gate, then for its in-neighbors $(b, c) = (\mathrm{in}_{1,i}(a), \mathrm{in}_{2,i}(a))$ we have

$$\mathrm{add}_i(a, b, c) = 1, \quad \mathrm{mult}_i(a, b, c) = 0,$$

and for all other (b, c), both predicates are 0. Hence the right-hand side reduces to

$$\widetilde{W}_{i+1}(b) + \widetilde{W}_{i+1}(c) = W_{i+1}(b) + W_{i+1}(c) = W_i(a) = \widetilde{W}_i(a).$$

The same reasoning holds when a is a multiplication gate, in which case only $\mathrm{mult}_i(a, b, c) = 1$ for the correct wiring, giving $\widetilde{W}_i(a) = \widetilde{W}_{i+1}(b)\widetilde{W}_{i+1}(c)$. Therefore, the identity holds for all Boolean z, and hence for all $z \in \mathbb{F}^{k_i}$. □

Interpretation. Lemma 6.1 algebraically encodes the local consistency of layer i with its inputs from layer $i + 1$. It serves as the basis for an application of the sum-check protocol, allowing the verifier to check correctness of each layer transition without explicitly evaluating all gates.

6.5 The GKR Protocol as an Iterative Application of Sum-Check

Lemma 6.1 provides a direct way to express the value of layer i as a polynomial in the values of layer $i+1$. This enables a recursive verification strategy: the verifier will check, layer by layer, that each claimed multilinear extension $\widetilde{W}_i$ satisfies its defining relation with respect to $\widetilde{W}_{i+1}$.

High-level structure. The protocol proceeds in d iterations, one per layer. At the beginning of iteration i, the prover claims that $\widetilde{W}_i(r_i)$ equals some field element v_i, where $r_i \in \mathbb{F}^{k_i}$ is a random point chosen by the verifier. The goal of that iteration is to reduce this claim to a new one about $\widetilde{W}_{i+1}(r_{i+1})$ for a randomly chosen $r_{i+1} \in \mathbb{F}^{k_{i+1}}$. The base case occurs at layer d, where the verifier can directly evaluate $\widetilde{W}_d(r_d)$ from the circuit inputs.

6.5.1 Initial Step

At the start, the verifier holds a function $D : \{0,1\}^{k_0} \to \mathbb{F}$ describing the claimed output layer values and wishes to check

$$\widetilde{D}(r_0) = \widetilde{W_0}(r_0)$$

for a random $r_0 \in \mathbb{F}^{k_0}$. Because V can efficiently compute $\widetilde{D}(r_0)$, correctness of the claim implies correctness of the circuit's output with high probability (by the Schwartz–Zippel lemma). However, V cannot compute $\widetilde{W_0}(r_0)$ directly; this is precisely what the GKR iteration aims to verify.

6.5.2 Defining the Target Polynomial for Sum-Check

From Lemma 6.1, we can write

$$\widetilde{W_i}(r_i) = \sum_{b,c \in \{0,1\}^{k_{i+1}}} f_{r_i}^{(i)}(b,c),$$

where

$$f_{r_i}^{(i)}(b,c) = \widetilde{\mathrm{add}}_i(r_i,b,c)\big(\widetilde{W}_{i+1}(b) + \widetilde{W}_{i+1}(c)\big) + \widetilde{\mathrm{mult}}_i(r_i,b,c)\big(\widetilde{W}_{i+1}(b)\widetilde{W}_{i+1}(c)\big). \tag{6.1}$$

The verifier thus needs to confirm that the sum of $f_{r_i}^{(i)}$ over all $(b,c) \in \{0,1\}^{2k_{i+1}}$ equals the claimed value $\widetilde{W_i}(r_i)$. This is an exact instance of the setting addressed by the sum-check protocol.

Application of sum-check. P and V engage in the sum-check protocol for the polynomial $f_{r_i}^{(i)}$, where:

$$H_i = \sum_{b,c \in \{0,1\}^{k_{i+1}}} f_{r_i}^{(i)}(b,c)$$

is the claimed sum. The verifier's checks in the sum-check protocol ensure that, except with probability at most $O(k_{i+1}/|\mathbb{F}|)$, the prover's claim about H_i is consistent with some particular evaluation of $f_{r_i}^{(i)}$ at a random point (b^*, c^*) in $\mathbb{F}^{2k_{i+1}}$.

After the sum-check protocol concludes, the verifier needs to verify a new claim:

$$f_{r_i}^{(i)}(b^*, c^*) = (\text{algebraic expression involving } \widetilde{W}_{i+1}(b^*) \text{ and } \widetilde{W}_{i+1}(c^*)).$$

However, this claim involves two distinct evaluations of $\widetilde{W}_{i+1}$ one at b^* and one at c^*—while the recursion requires reducing to a single evaluation.

6.5.3 Reduction to a Single Evaluation via Line Interpolation

To unify the two points (b^*, c^*), the verifier defines an affine line

$$\ell(\alpha) = b^* + \alpha(c^* - b^*) \in \mathbb{F}^{k_{i+1}},$$

satisfying $\ell(0) = b^*$ and $\ell(1) = c^*$. The prover then provides a univariate polynomial

$$q(\alpha) = \widetilde{W}_{i+1}(\ell(\alpha)),$$

which is of degree at most k_{i+1}, since $\widetilde{W}_{i+1}$ is multilinear.

The verifier checks consistency at the endpoints:

$$q(0) = \widetilde{W}_{i+1}(b^*), \quad q(1) = \widetilde{W}_{i+1}(c^*),$$

as implied by the transcript of the sum-check. Then V samples a random $\alpha^* \in \mathbb{F}$ and defines

$$r_{i+1} := \ell(\alpha^*).$$

The verifier now needs to verify only the single claim that

$$q(\alpha^*) = \widetilde{W}_{i+1}(r_{i+1}).$$

This becomes the input to the next iteration of the protocol.

Intuitively, this linear interpolation step is what makes the GKR protocol recursive. At each layer, the verifier reduces a claim about one layer's multilinear extension to a claim about the next layer's extension, but at a single random point. The recursion terminates when the verifier reaches the input layer, whose values are known explicitly.

6.6 Final Layer and Termination

At the final layer d, $\widetilde{W}_d$ encodes the circuit inputs. Because V knows all input values, it can evaluate $\widetilde{W}_d(r_d)$ directly in time $O(2^{k_d})$, which is linear in the number of input gates and typically much smaller than the total circuit size. If this final check passes, V accepts; otherwise, it rejects.

6.7 Completeness and Soundness

The correctness and complexity guarantees stated in this section follow the original analysis of the GKR protocol by Goldwasser, Kalai, and Rothblum [GKR15], and are presented here in a streamlined and self-contained form.

Theorem 6.2 (Soundness and completeness of GKR)**.** *Let C be a layered arithmetic circuit with fan-in two and depth d. Let S denote the total number of gates and $\mathbb{F}$ the underlying field. Then the GKR protocol described above is an interactive proof system for verifying $y = C(x)$ with:*

- ***Perfect completeness:*** *an honest prover always causes V to accept.*
- ***Soundness error:*** *at most $O\big(\sum_{i=0}^{d-1} \frac{k_{i+1}}{|\mathbb{F}|}\big)$, dominated by the sum-check invocations.*

Proof. Completeness. If the prover behaves honestly, then at every layer i, the polynomial $f_{r_i}^{(i)}$ defined in (6.1) satisfies

$$\sum_{b,c\in\{0,1\}^{k_{i+1}}} f_{r_i}^{(i)}(b,c) = \widetilde{W}_i(r_i).$$

Each invocation of the sum-check protocol thus succeeds, and the line interpolation check maintains consistency of all random evaluations down to layer d, where V computes the final check directly. Hence V always accepts.

Soundness. If the prover's claim is false, then at some layer i, the polynomial $f_{r_i}^{(i)}$ used in the sum-check invocation differs from the true polynomial corresponding to the honest relation. By the soundness of the sum-check protocol, the probability that such a discrepancy passes undetected is at most $(2k_{i+1})/|\mathbb{F}|$. Furthermore, each line interpolation check introduces at most another $1/|\mathbb{F}|$ chance of error. By a union bound over all layers, the total soundness error is at most $O(\sum_i k_{i+1}/|\mathbb{F}|)$. $\square$

Interpretation. The GKR protocol amplifies the principle of the sum-check protocol to an entire circuit. At each step, it transforms the verification of a large structured computation into a small number of random polynomial checks. The final result is a succinct, layer-by-layer proof of correct circuit evaluation.

6.8 Complexity and Efficiency

Verifier. In iteration i, the verifier participates in a sum-check protocol over a polynomial in $2k_{i+1}$ variables of individual degree at most 2. Each such invocation requires $2k_{i+1}$ rounds and $O(k_{i+1})$ field operations. Across all d layers, the total verifier time is

$$O(n + d \log S + t + S_0),$$

where:

- n is the input size,
- t is the cost of evaluating the wiring predicates add_i and mult_i,
- S_0 is the number of output gates.

The verifier's memory footprint and total communication are both $O(S_0 + d \log S)$ field elements.

Prover. At layer i, the prover must evaluate $f_{r_i}^{(i)}$ at all Boolean assignments to its $2k_{i+1}$ variables, which can be done with $O(2^{2k_{i+1}})$ field operations. Summed over all layers, the total prover work is $O(S \log S)$ for circuits with regular wiring (and $O(S^3)$ in the worst case if wiring is arbitrary). Thus, the prover's cost is nearly linear in the circuit size.

Chapter 7

Plookup

7.1 Motivation and Overview

Many computations require verifying that certain witness values belong to a fixed, public set of allowed values. A simple example is enforcing that a variable x represents a Boolean value, which amounts to checking that $x \in \{0, 1\}$. More generally, one may wish to enforce *range constraints*, ensuring that a witness lies within a prescribed interval, or membership constraints asserting inclusion in an arbitrary finite set.

These constraints arise naturally in a variety of settings, from ensuring that variables encode valid bits or small integers, to verifying table-driven computations and conditional logic. For example, given a function specified by a lookup table

$$T = \{(a, b) : b = f(a)\},$$

a prover may need to convince the verifier that each witness pair (a_i, b_i) used in the computation appears in T, without revealing additional information about the witness.

Naively enforcing such relations through algebraic constraints would require a polynomial for each table entry, resulting in large circuits. **Lookup arguments** allow these membership checks to be encoded succinctly and verified with logarithmic overhead. They have thus become a core building block in modern SNARKs such as PLONK, Halo 2, and Lasso.

This chapter presents the **Plookup protocol** [GWC20], a polynomial protocol that

efficiently proves a lookup relation of the form

$$\mathbf{f} \subseteq \mathbf{t},$$

where $\mathbf{f}$ is a vector of witness values and $\mathbf{t}$ is a fixed lookup table. The protocol demonstrates this inclusion using only polynomial relations over a multiplicative subgroup of the field, leveraging a grand product argument reminiscent of the permutation check in PLONK.

7.2 Lookup Relations and Notation

Let $\mathbf{f} = (f_1, \ldots, f_n) \in \mathbb{F}^n$ represent the witness values that must appear in the lookup table $\mathbf{t} = (t_1, \ldots, t_{n+d}) \in \mathbb{F}^{n+d}$. The inclusion relation

$$\mathbf{f} \subseteq \mathbf{t}$$

means that every f_i occurs somewhere in $\mathbf{t}$, possibly with repetition. To encode this relation algebraically, we introduce an auxiliary vector $\mathbf{s} = (s_1, \ldots, s_{n+d})$, which is a *sorted permutation* of $\mathbf{f} \cup \mathbf{t}$ according to the order of the table $\mathbf{t}$. The role of the Plookup protocol is to check, using polynomials, that: 1. $\mathbf{f} \subseteq \mathbf{t}$, and 2. $\mathbf{s}$ represents a consistent interleaving of $\mathbf{f}$ and $\mathbf{t}$.

Throughout, let $H = \{g, g^2, \ldots, g^N\}$ denote a multiplicative subgroup of $\mathbb{F}$ of order N, over which all polynomials will be evaluated.

7.3 Encoding the Lookup Relation with Polynomials

The core of Plookup is to translate the discrete inclusion check into an algebraic identity between two bivariate polynomials. Define $F(\beta, \gamma)$ and $G(\beta, \gamma)$ as follows:

$$F(\beta, \gamma) = (1 + \beta)^n \prod_{i=1}^{n} (\gamma + f_i + \beta l_i) \prod_{i=n+1}^{n+d} (\gamma + \beta l_i), \tag{7.1}$$

$$G(\beta, \gamma) = \prod_{i=1}^{n+d} (\gamma + t_i + \beta s_i). \tag{7.2}$$

Intuitively, F encodes the witness values f_i together with placeholder terms, while G encodes the table entries t_i and their sorted permutation s_i. The random coefficients $\beta, \gamma \in \mathbb{F}$ bind both structures, ensuring that any mismatch in inclusion or order alters the resulting polynomial.

The claim that $\mathbf{f} \subseteq \mathbf{t}$ and that $\mathbf{s}$ is correctly sorted therefore translates into the polynomial identity

$$F(\beta, \gamma) \equiv G(\beta, \gamma),$$

which must hold as an equality of polynomials in $\mathbb{F}[\beta, \gamma]$, not merely for specific evaluations.

7.4 Correctness of the Encoding

Lemma 7.1. *The equality $F(\beta, \gamma) \equiv G(\beta, \gamma)$ holds if and only if:*

1. $\mathbf{f} \subseteq \mathbf{t}$, *and*

2. $\mathbf{s}$ *is a correctly sorted interleaving of* $\mathbf{f}$ *within* $\mathbf{t}$.

Proof. ($\Rightarrow$) Assume $F \equiv G$. Then their roots as polynomials in γ coincide. Each factor of F has the form $(\gamma + f_i + \beta l_i)$ or $(\gamma + \beta l_i)$, while each factor of G has the form $(\gamma + t_i + \beta s_i)$. For the equality to hold, every f_i in F must match some t_j in G; hence $\mathbf{f} \subseteq \mathbf{t}$. Moreover, any mismatch in order would yield a distinct linear factor, so $\mathbf{s}$ must follow the ordering of $\mathbf{t}$.

($\Leftarrow$) Conversely, if $\mathbf{f} \subseteq \mathbf{t}$ and $\mathbf{s}$ respects the order of $\mathbf{t}$, then each factor $(\gamma + f_i + \beta l_i)$ of F pairs with some $(\gamma + t_j + \beta s_j)$ of G, possibly after reindexing via $\mathbf{s}$. The remaining factors $(\gamma + \beta l_i)$ in F correspond exactly to unmatched t_j terms. Hence all linear factors coincide, and $F \equiv G$. $\square$

Intuitively, F and G serve as compact encodings of multisets. If they differ, their roots differ, so for random (β, γ) the evaluations disagree with overwhelming probability. Thus, verifying $F(\beta, \gamma) = G(\beta, \gamma)$ for random challenges provides a succinct probabilistic test of the lookup relation.

7.5 From Algebraic Identity to Protocol

Lemma 7.1 suggests a natural interactive proof: the prover can convince the verifier that $F \equiv G$ by demonstrating the algebraic consistency of the products defining these polynomials. To avoid evaluating large products directly, Plookup introduces an auxiliary polynomial $Z(X)$ whose values encode a running quotient between the partial products of F and G. This converts the equality $F = G$ into a sequence of local constraints over the subgroup H.

7.6 The Plookup Protocol

Following the equivalence in Lemma 7.1, the prover's goal is to convince the verifier that the product encodings F and G coincide. Instead of comparing full products directly, Plookup introduces a *grand product polynomial* $Z(X)$ that tracks the cumulative ratio between the partial products of F and G across a subgroup $H = \{g, g^2, \ldots, g^N\}$.

7.6.1 Witness Polynomials

The prover encodes the sequences

$$\mathbf{f} = (f_1, \ldots, f_n), \qquad \mathbf{t} = (t_1, \ldots, t_{n+d}), \qquad \mathbf{s} = (s_1, \ldots, s_{n+d})$$

as univariate polynomials over H:

$$h_1(g^i) = s_i, \qquad h_2(g^i) = s_{N+i}, \quad i = 1, \ldots, N.$$

The polynomials $h_1, h_2 \in \mathbb{F}[X]$ interpolate the sorted witness $\mathbf{s}$ and its shifted copy, which will be related through the verifier's consistency checks.

7.6.2 Verifier Challenges

The verifier samples random field elements $\beta, \gamma \in \mathbb{F}$ and sends them to the prover. These challenges randomize the linear factors of F and G, ensuring that any incorrect claim about inclusion or sorting will alter the resulting product with high probability. This randomization follows the same principle as in the permutation argument of PLONK.

7.6.3 The Grand Product Polynomial

Given (β, γ), the prover constructs a polynomial $Z(X)$ over H defined recursively by:

$$Z(g) = 1, \qquad \frac{Z(g^{i+1})}{Z(g^i)} = \frac{(1+\beta+f_i)(1+\beta+h_1(g^i)+h_2(g^{i+1}))}{(1+\beta+t_i+\beta g^i)}, \quad i = 1, \ldots, N-1,$$

and satisfying $Z(g^N) = 1$. Each ratio $Z(g^{i+1})/Z(g^i)$ captures the local equality between the corresponding factors of F and G.

Telescoping property. If all ratios are correct, the product of the right-hand sides over $i = 1, \ldots, N-1$ equals 1, implying

$$Z(g^N) = Z(g) \prod_{i=1}^{N-1} \frac{Z(g^{i+1})}{Z(g^i)} = 1.$$

This telescoping effect ensures that a single violation at any index i propagates to an incorrect final value of $Z(g^N)$, which the verifier can detect.

7.6.4 Verifier Checks

The verifier must confirm that $Z(X)$ satisfies the intended recurrence and boundary conditions over H. This is encoded through the following polynomial constraints:

$$L_1(X)\,(Z(X) - 1) = 0, \tag{7.3}$$

$$L_N(X)\,(Z(X) - 1) = 0, \tag{7.4}$$

$$L_N(X)\,(h_2(X) - h_2(gX)) = 0, \tag{7.5}$$

$$Z(gX)\,(1+\beta+f(X))\,(1+\beta+h_1(X)+h_2(gX)) = Z(X)\,(1+\beta+t(X)+\beta\, g(X)). \tag{7.6}$$

Here $L_1(X)$ and $L_N(X)$ denote the Lagrange basis polynomials that evaluate to 1 at $X = g$ and $X = g^N$, respectively, and to 0 elsewhere.

Interpretation of constraints.

- (7.3) enforces the initialization $Z(g) = 1$;
- (7.4) enforces the final boundary $Z(g^N) = 1$;

- (7.5) ensures the wrap-around consistency of the shifted polynomial h_2;
- (7.6) enforces the correct multiplicative recurrence between successive evaluations of Z.

If all equalities hold over the entire subgroup, the inclusion relation $\mathbf{f} \subseteq \mathbf{t}$ must be satisfied.

7.7 Completeness and Soundness

The correctness and soundness guarantees stated in this section follow the analysis of the Plookup protocol introduced by Eagen, Fiore, Gabizon, and Meneghetti [GWC20], and are presented here in a self-contained form.

Theorem 7.2 (Correctness of Plookup)**.** *Let $H \subset \mathbb{F}$ be a multiplicative subgroup of order N. If the prover and verifier execute the protocol with random challenges (β, γ), then:*

1. *If $\mathbf{f} \subseteq \mathbf{t}$ and $\mathbf{s}$ is a correctly sorted permutation, the verifier always accepts.*
2. *Otherwise, the probability that an incorrect witness passes all checks is at most $O(1/|\mathbb{F}|)$.*

Proof. **Completeness.** If $\mathbf{f} \subseteq \mathbf{t}$ and $\mathbf{s}$ is correctly sorted, then by Lemma 7.1, the polynomial identity $F \equiv G$ holds. For each i, the recurrence defining Z satisfies

$$\frac{Z(g^{i+1})}{Z(g^i)} = \frac{(1+\beta+f_i)\,(1+\beta+h_1(g^i)+h_2(g^{i+1}))}{(1+\beta+t_i+\beta g^i)}.$$

Since $F \equiv G$, every ratio equals 1, and the product telescopes:

$$Z(g^N) = Z(g)\prod_{i=1}^{N-1}\frac{Z(g^{i+1})}{Z(g^i)} = 1.$$

Hence all checks (7.3)–(7.6) hold exactly, and the verifier accepts with certainty.

Soundness. If either inclusion or sorting fails, at least one factor in (7.6) differs from the honest recurrence. Define

$$E(X) = Z(gX)\,(1+\beta+f(X))\,(1+\beta+h_1(X)+h_2(gX)) - Z(X)\,(1+\beta+t(X)+\beta g(X)).$$

When the prover cheats, $E(X)$ is a nonzero polynomial of bounded degree. By the Schwartz–Zippel lemma, for random (β, γ), the equality $E(X) = 0$ over H holds with probability at most $1/|\mathbb{F}|$. Thus, incorrect witnesses are detected except with negligible probability. $\square$

7.8 Complexity

All polynomials f, t, h_1, h_2, Z have degree at most N. Each verifier check uses a constant number of field operations, so with polynomial commitments the verifier's work is logarithmic in N. The prover constructs $Z(X)$ in $O(N)$ time by sequentially applying the recurrence, linear in the table size.

Chapter 8

Logup

8.1 Motivation

Lookup arguments verify that witness values appear in a fixed table of valid entries. In the previous chapter, we saw how PLOOKUP enforces this condition multiplicatively: the prover and verifier check that two grand products, one derived from the witness column, one from the table, evolve identically under a random linear combination. While this approach is sound and efficient, it introduces nonlinear constraints and relies on additional random columns to maintain permutation consistency.

The **Logup** protocol, proposed by Haböck [Hab23], eliminates this multiplicative layer entirely by working with *logarithmic derivatives*. Instead of verifying that two products of linear factors coincide, Logup verifies that their logarithmic derivatives are equal. This change turns a multiplicative constraint into an additive one, making it compatible with sumcheck-based constructions such as GKR and Lasso.

8.2 From Grand Products to Logarithmic Derivatives

Let $\mathbf{f} = (f_1, \ldots, f_n)$ denote the prover's witness values and $\mathbf{t} = (t_1, \ldots, t_{n+d})$ the public lookup table, both over a finite field $\mathbb{F}$. The prover wishes to convince the verifier that each f_i appears among the table entries t_j, that is,

$$\{f_1, \ldots, f_n\} \subseteq \{t_1, \ldots, t_{n+d}\}.$$

Encoding the sets as polynomials. Define the univariate polynomials

$$F(X) = \prod_{i=1}^{n}(X - f_i), \qquad T(X) = \prod_{j=1}^{n+d}(X - t_j).$$

If $\mathbf{f} \subseteq \mathbf{t}$, then $F(X)$ divides $T(X)$, so that

$$\frac{T(X)}{F(X)} = R(X)$$

for some polynomial R collecting the remaining factors.

Taking formal derivatives yields the *logarithmic derivative identity*

$$\frac{F'(X)}{F(X)} = \sum_{i=1}^{n} \frac{1}{X - f_i}, \qquad \frac{T'(X)}{T(X)} = \sum_{j=1}^{n+d} \frac{1}{X - t_j}.$$

Hence inclusion $\mathbf{f} \subseteq \mathbf{t}$ is equivalent to the existence of nonnegative multiplicities m_j such that

$$\sum_{i=1}^{n} \frac{1}{X - f_i} = \sum_{j=1}^{n+d} \frac{m_j}{X - t_j}. \tag{8.1}$$

Lemma 8.1 (Equality via logarithmic derivatives)**.** *Let $\mathbb{F}$ be a field of characteristic $p > n + d$. Then for any sequences $(a_i)_{i=1}^{n}$ and $(b_j)_{j=1}^{m}$ in $\mathbb{F}$,*

$$\prod_{i=1}^{n}(X - a_i) = c\prod_{j=1}^{m}(X - b_j)^{m_j} \quad \Longleftrightarrow \quad \sum_{i=1}^{n} \frac{1}{X - a_i} = \sum_{j=1}^{m} \frac{m_j}{X - b_j}$$

for some constant $c \in \mathbb{F}^{\times}$.

Proof. Differentiating $\log(F(X)) - \log(T(X)^{\mathbf{m}})$ gives the right-to-left direction. Conversely, if the rational functions coincide, then their ratio $Q(X) = F(X)/T(X)^{\mathbf{m}}$ has derivative zero and must be constant, so equality holds up to scaling. □

Intuition. In Plookup, the verifier checks multiplicative ratios across domain elements. In Logup, multiplicative structure is replaced by additive rational sums. This linearization allows lookup checks to be expressed as ordinary polynomial equalities, removing the need for grand products or batched permutation arguments.

8.3 The Univariate Logup Protocol

Let $H = \{g, g^2, \ldots, g^N\}$ be a multiplicative subgroup of $\mathbb{F}^\times$ of size $N \geq n+d$. The prover is given evaluation vectors

$$\mathbf{f} = (f_1, \ldots, f_n), \quad \mathbf{t} = (t_1, \ldots, t_{n+d}),$$

and commits to corresponding polynomials $f(x), t(x)$ defined over H. The goal is to prove that the range of f is contained in that of t.

8.3.1 Sampling a Random Evaluation Point

The verifier samples a random $\alpha \leftarrow \mathbb{F}$. Evaluating the logarithmic derivative identity (8.1) at $X = \alpha$ gives

$$\sum_{i=1}^{n} \frac{1}{\alpha - f_i} = \sum_{j=1}^{n+d} \frac{m_j}{\alpha - t_j}.$$

Define the helper functions

$$h_f(x_i) = \frac{1}{\alpha - f_i}, \qquad h_t(x_j) = \frac{1}{\alpha - t_j}.$$

Then the inclusion condition is equivalent to

$$\sum_{i=1}^{n} h_f(x_i) = \sum_{j=1}^{n+d} m_j h_t(x_j). \tag{8.2}$$

8.3.2 Expressing the Relation on the Evaluation Domain

Extending these to polynomials $h_f(x), h_t(x)$ over H, equation (8.2) can be rewritten as a single constraint over the subgroup:

$$\sum_{x \in H} \big(h_f(x) - m(x)h_t(x)\big) = 0,$$

where $m(x)$ encodes the multiplicity with which table element $t(x)$ is used. This is a linear constraint that can be verified through the standard sumcheck protocol.

Protocol sketch.

1. The verifier samples $\alpha \in \mathbb{F}$ at random.

2. The prover constructs helper polynomials $h_f(x), h_t(x)$ satisfying $(\alpha - f(x))h_f(x) = 1$ and $(\alpha - t(x))h_t(x) = 1$ on H.

3. The prover and verifier engage in a sumcheck protocol to verify that

$$\sum_{x \in H} \big(h_f(x) - m(x)h_t(x)\big) = 0.$$

4. The verifier performs one final evaluation check on the claimed sumcheck transcript.

The key observation is that each step is linear in the witness values. All multiplicative dependencies from Plookup's grand products are replaced by additive relations between evaluations of h_f and h_t. This is the *telescoping property* of Logup: the ratios that would multiply across domain elements in Plookup now add up and cancel directly in the rational sum.

Remark on telescoping. In Plookup, correctness follows from the telescoping of consecutive grand products $Z(g^{i+1})/Z(g^i)$. In Logup, telescoping arises additively: each term $\frac{1}{\alpha - f_i}$ contributes linearly to the rational identity, and any mismatch creates a nonzero residue that the sumcheck detects with high probability. Thus the same cancellation phenomenon occurs, but in additive rather than multiplicative form.

8.3.3 Completeness and Soundness

Theorem 8.2 (Completeness and soundness of Logup)**.** *Let $\mathbb{F}$ be a finite field and $H \subset \mathbb{F}$ a multiplicative subgroup of size N. Let $\mathbf{f}, \mathbf{t} \in \mathbb{F}^N$ denote, respectively, the witness and table values, and let $f(x), t(x)$ be their corresponding polynomials over H. Then the Logup protocol described above is an interactive proof of inclusion $\mathbf{f} \subseteq \mathbf{t}$ with:*

- ***Perfect completeness:*** *if $\mathbf{f} \subseteq \mathbf{t}$, an honest prover always causes the verifier to accept;*

- ***Soundness error:*** *at most $O(1/|\mathbb{F}|)$, assuming $\deg(f), \deg(t) < |H|$.*

Proof. If $\mathbf{f} \subseteq \mathbf{t}$, then for each $\alpha \in \mathbb{F}$,

$$\sum_i \frac{1}{\alpha - f_i} = \sum_j \frac{m_j}{\alpha - t_j},$$

so the polynomial relation

$$P(X) = \sum_{x \in H} (h_f(x) - m(x)h_t(x))$$

vanishes identically. Hence the verifier accepts with probability 1.

Conversely, if $\mathbf{f} \not\subseteq \mathbf{t}$, then there exists some $x^* \in H$ such that $h_f(x^*) \neq m(x^*)h_t(x^*)$, so $P(X)$ is a nonzero polynomial of degree at most $\deg(h_f)+\deg(h_t)$. By the Schwartz–Zippel lemma, a random choice of α detects this inequality except with probability $\leq (\deg P)/|\mathbb{F}| = O(1/|\mathbb{F}|)$. □

8.4 Complexity and Efficiency

The complexity bounds in this section follow the analysis presented in Haböck's Logup construction [Hab23], restated here in a streamlined and self-contained form.

Verifier. The verifier samples one random field element α and participates in a short sumcheck over a degree-$O(1)$ polynomial in the field. The total number of field operations is $O(\log N)$, dominated by polynomial commitment openings. The verifier's state and communication cost are both constant.

Prover. The prover computes $h_f(x) = 1/(\alpha - f(x))$ and $h_t(x) = 1/(\alpha - t(x))$ for all $x \in H$. This requires $O(N)$ field operations linear in the size of the witness and table and one sumcheck transcript of logarithmic length. Hence the total prover work remains $O(N)$, identical to Plookup's.

8.5 Comparison with Plookup

We now summarize the essential differences between the two lookup constructions, highlighting how Logup replaces multiplicative reasoning with linear relations.

Core algebraic relation. Plookup enforces equality of *grand products*, verifying that consecutive multiplicative ratios coincide over the evaluation domain. Logup instead enforces equality of *logarithmic derivatives*, comparing additive rational sums of the form $\sum_i 1/(\alpha - f_i)$ and $\sum_j 1/(\alpha - t_j)$. The multiplicative telescoping in Plookup becomes additive telescoping in Logup.

Constraint structure. In Plookup, the defining relation $Z(gx)(1+\beta+f(x))(1+\beta+h_1(x)+h_2(gx)) = Z(x)(1+\beta+t(x)+\beta g(x))$ is nonlinear in its variables. Logup replaces this with the linear identity (8.2), expressible as a single bilinear constraint between h_f and h_t.

Verifier work. Plookup requires either $O(\log N)$ openings or a batched product check. Logup achieves the same guarantee with a constant number of linear checks.

Composition. Because Logup's constraints are linear, they compose directly with protocols like GKR or sumcheck. Plookup's multiplicative structure, in contrast, requires nonlinear combination gadgets when embedded in algebraic proof systems.

8.6 Toward the Multivariate Case

While the exposition above focuses on univariate domains $H \subset \mathbb{F}$, the same logarithmic-derivative principle extends naturally to multivariate settings. Given polynomials defined over $\{0,1\}^\ell$, we can define partial derivatives with respect to each coordinate, and replace the univariate rational identity by a multivariate one:

$$\sum_i \frac{1}{\alpha - f(\mathbf{x}_i)} = \sum_j \frac{m_j}{\alpha - t(\mathbf{x}_j)}.$$

Chapter 9

Logup with GKR

Building on the ideas of the Logup protocol introduced by Haböck [Hab23], this chapter presents a formulation that integrates Logup directly within the GKR framework of Goldwasser, Kalai, and Rothblum.

The construction follows the perspective of Haböck's technical note, which interprets lookup verification as a problem of rational sum evaluation, and extends it using the multilinear proof machinery developed in GKR. This combination eliminates the need for auxiliary helper polynomials and transforms the logarithmic derivative identity of Logup into a succinct *fractional sumcheck*. The result is a lookup protocol that remains linear in the witness values while being fully compatible with sumcheck-based proof systems such as Lasso.

9.1 Notation

We work over a finite field $\mathbb{F}$ of characteristic larger than 2, with multiplicative group $\mathbb{F}^*$. For any integer $n \geq 1$, denote by $H_n = \{\pm 1\}^n$ the n-dimensional Boolean hypercube, viewed as a subgroup of $(\mathbb{F}^*)^n$. This choice of representation is equivalent to the Boolean hypercube $\{0,1\}^n$ used in previous chapters, via the standard affine mapping between $\{0,1\}$ and $\{\pm 1\}$, and is adopted here for its algebraic convenience.

A *multilinear polynomial* $p(X_1, \ldots, X_n)$ over $\mathbb{F}$ is a polynomial such that each variable has degree at most one:

$$\deg_{X_i}(p) \leq 1 \quad \forall\, i \in [n].$$

We use vector notation $p(\mathbf{X})$ for $p(X_1, \ldots, X_n)$ and may decompose it as

$$p((X_1, \ldots, X_k), (X_{k+1}, \ldots, X_n))$$

when necessary.

The *Lagrange kernel* on H_n is defined as

$$L_n(\mathbf{X}, \mathbf{Y}) = 2^{-n} \prod_{i=1}^{n} (1 + X_i Y_i),$$

a multilinear polynomial that evaluates to 1 if $\mathbf{X} = \mathbf{Y}$ and to 0 otherwise over H_n. All multilinear polynomials in this discussion will be given in this Lagrange representation, that is, by their values over H_n. Protocols are therefore described as *Lagrange IOPs*, where oracles correspond to value tables over H_n and are accessed through tensor (evaluation) queries.

9.2 From Logup to Fractional Sumchecks

Because the Logup identity is linear in the witness values, it can be verified as a sum over rational functions. This leads to the *fractional sumcheck*, which expresses the same inclusion condition in a form directly compatible with the GKR framework.

In the multivariate version of Logup, the prover holds $M \geq 1$ witness columns given by multilinear polynomials

$$w_1(\mathbf{X}), \ldots, w_M(\mathbf{X}),$$

and must prove that all values lie in the table defined by another multilinear polynomial $t(\mathbf{X})$, whose evaluations over H_n are pairwise distinct. To encode multiplicities, define

$$m(\mathbf{x}) = \sum_{i=1}^{M} \left| \{\mathbf{y} \in H_n : w_i(\mathbf{y}) = t(\mathbf{x})\} \right|, \quad \forall \mathbf{x} \in H_n.$$

The inclusion relation can be written as the polynomial identity

$$\prod_{i=1}^{M} \prod_{\mathbf{x} \in H_n} (X - w_i(\mathbf{x})) = \prod_{\mathbf{x} \in H_n} (X - t(\mathbf{x}))^{m(\mathbf{x})}. \tag{9.1}$$

This equality is equivalent to the equality of logarithmic derivatives:

$$\sum_{i=1}^{M} \sum_{\mathbf{x} \in H_n} \frac{1}{X - w_i(\mathbf{x})} = \sum_{\mathbf{x} \in H_n} \frac{m(\mathbf{x})}{X - t(\mathbf{x})}. \tag{9.2}$$

Evaluating both sides at a random $\alpha \in \mathbb{F}$ yields the central *fractional sumcheck equation*:

$$\sum_{\mathbf{x} \in H_n} \left(\frac{m(\mathbf{x})}{\alpha - t(\mathbf{x})} - \sum_{i=1}^{M} \frac{1}{\alpha - w_i(\mathbf{x})} \right) = 0. \tag{9.3}$$

Earlier variants of Logup reduced this rational identity to a polynomial one by introducing auxiliary helper functions

$$h(\mathbf{X}) = \frac{1}{\alpha - t(\mathbf{X})}, \quad h_i(\mathbf{X}) = \frac{1}{\alpha - w_i(\mathbf{X})},$$

and verifying them through domain relations such as $(\alpha - t(\mathbf{X}))h(\mathbf{X}) = 1$ over H_n. While effective, this method requires $M + 1$ additional committed polynomials, which increases prover cost and circuit complexity.

9.3 GKR for Fractional Sumchecks

To evaluate the fractional sum succinctly, we can invoke the Goldwasser–Kalai–Rothblum (GKR) protocol, which efficiently verifies layered arithmetic computations such as rational sums over the Boolean hypercube. Instead of introducing helper polynomials, the prover can use GKR to prove Equation (9.3) directly as a sum over fractions. This is achieved by constructing a layered arithmetic circuit that computes

$$S = \sum_{\mathbf{x} \in H_n} \frac{p(\mathbf{x})}{q(\mathbf{x})}$$

in *projective coordinates* $(a, b) \in \mathbb{F}^2$, representing fractions a/b.

9.3.1 Projective addition

For two fractions (a_0, b_0) and (a_1, b_1), define their sum in projective form as

$$(a_0, b_0) +_F (a_1, b_1) = (a_0 b_1 + a_1 b_0,\ b_0 b_1).$$

This operation propagates naturally through a binary tree structure. Each layer k of the circuit corresponds to partial sums over H_k, with each node $\mathbf{x} \in H_k$ carrying a pair $(p_k(\mathbf{x}), q_k(\mathbf{x}))$ that represents the cumulative sum of fractions below it:

$$\frac{p_k(\mathbf{x})}{q_k(\mathbf{x})} = \sum_{\mathbf{y} \in H_{n-k}} \frac{p(\mathbf{x}, \mathbf{y})}{q(\mathbf{x}, \mathbf{y})}.$$

At the top layer ($k = 0$), the pair (p_0, q_0) encodes the total sum $S = p_0/q_0$.

9.3.2 Layer relations

For each layer k and node $\mathbf{x} \in H_k$, the recursive relations are

$$\begin{aligned} p_k(\mathbf{x}) &= p_{k+1}(\mathbf{x}, +1) q_{k+1}(\mathbf{x}, -1) + p_{k+1}(\mathbf{x}, -1) q_{k+1}(\mathbf{x}, +1), \\ q_k(\mathbf{x}) &= q_{k+1}(\mathbf{x}, +1) q_{k+1}(\mathbf{x}, -1). \end{aligned}$$

At each layer the verifier must be convinced that these identities hold for all $\mathbf{x} \in H_k$. The GKR protocol performs this check layer by layer using multilinear extensions and random linear combinations, reducing all checks to a single final evaluation.

9.4 Embedding into Logup

The fractional sumcheck of Equation (9.3) can be written as

$$\sum_{\mathbf{x} \in H_n} \frac{p(\mathbf{x})}{q(\mathbf{x})} = 0,$$

where the polynomials p, q in $n + k$ variables are defined by

$$\begin{aligned} p(\mathbf{X}, \mathbf{Y}) &= L_k(\mathbf{Y}, \mathbf{1})\, m(\mathbf{X}) - \sum_{\mathbf{y} \in H_k \setminus \{\mathbf{1}\}} L_k(\mathbf{Y}, \mathbf{y}), \\ q(\mathbf{X}, \mathbf{Y}) &= L_k(\mathbf{Y}, \mathbf{1})\, (\alpha - t(\mathbf{X})) + \sum_{\mathbf{y} \in H_k \setminus \{\mathbf{1}\}} L_k(\mathbf{Y}, \mathbf{y})\, (\alpha - w_{i(\mathbf{y})}(\mathbf{X})), \end{aligned}$$

with $\mathbf{1} = (1, \ldots, 1)$ and $i(\mathbf{y})$ an enumeration of $\{1, \ldots, M\}$. Applying the GKR protocol to this expression proves the fractional identity directly, and the verifier only needs to query $t(\mathbf{r})$, $m(\mathbf{r})$, and $w_i(\mathbf{r})$ for a single random point $\mathbf{r} \in \mathbb{F}^n$.

9.5 Soundness and Complexity

The soundness bound and complexity estimates in this section follow the analysis of Haböck's Logup construction and its integration with sumcheck-based verification frameworks such as GKR [Hab23; GKR15], and are restated here for completeness.

Proposition 9.1 (Soundness of Logup–GKR)**.** *Let $M \geq 1$ and $\mathrm{char}(\mathbb{F}) > M \cdot 2^n$. The soundness error ε for Logup with GKR-based fractional sumchecks satisfies*

$$\varepsilon \leq \frac{(M+1)2^n - 1}{|\mathbb{F}| - |H|} + \varepsilon_{\mathrm{GKR}}(|H_{n+m}|),$$

where $m = \lceil \log_2(M+1) \rceil$ and $\varepsilon_{\mathrm{GKR}}$ is the soundness error of the GKR protocol over a domain of size $|H_{n+m}|$.

The dominant term arises from the reduction of the fractional identity to the sumcheck form. As before, the random challenge α is sampled outside H to ensure completeness.

Prover cost. The arithmetic cost for the GKR fractional sumcheck over H_n is approximately

$$|H_n| \cdot (43M + 29A),$$

where M and A denote field multiplications and additions. This is roughly double the raw arithmetic cost of the elementary Logup variant, which is $|H_n| \cdot (19M + 16A)$, but it removes all $M + 1$ helper commitments. Since commitment overhead dominates in elliptic-curve and FRI-based schemes, this reduction leads to significant savings in practice, typically between two and ten times depending on the proof system.

Chapter 10

Spark: A Building Block of the Lasso Protocol

10.1 Introduction

Lasso is a lookup argument that builds upon two recent constructions: *Spark* and *Surge*.

Spark is a commitment scheme for *sparse multilinear* polynomials. **Surge** generalizes Spark to enable proofs of *structured inner products*.

By combining these two ideas, Lasso provides a simple and efficient lookup argument. It allows a prover to demonstrate that every entry of a committed vector a lies in a (possibly public) table t, expressed algebraically as an equality between their multilinear extensions.

Spark.

Let $\ell \in \mathbb{N}$ and let $\mathbb{F}$ be a finite field. An ℓ-variate polynomial $p : \mathbb{F}^\ell \to \mathbb{F}$ is *multilinear* if it has degree at most one in each variable.

For a function $f : \{0,1\}^\ell \to \mathbb{F}$, its *multilinear extension* (MLE), denoted $\tilde{f} : \mathbb{F}^\ell \to \mathbb{F}$, is the unique multilinear polynomial satisfying

$$\tilde{f}(x) = f(x) \quad \text{for all } x \in \{0,1\}^\ell.$$

An explicit expression for $\tilde{f}$ is:

$$\tilde{f}(x_1, \ldots, x_\ell) \;=\; \sum_{z \in \{0,1\}^\ell} f(z) \cdot L_z(x_1, \ldots, x_\ell),$$

where, for $z = (z_1, \ldots, z_\ell)$,

$$L_z(x_1, \ldots, x_\ell) \;=\; \prod_{i=1}^{\ell} \big(x_i z_i + (1 - x_i)(1 - z_i)\big).$$

Equivalently, $L_z(x)$ is the multilinear extension of the Boolean equality function $\mathsf{eq}(x, z)$. A vector $u \in \mathbb{F}^N$ can be viewed as a function $u : \{0,1\}^{\log N} \to \mathbb{F}$, and hence it admits an MLE $\tilde{u} : \mathbb{F}^{\log N} \to \mathbb{F}$. We say u is *m-sparse* if only $m \ll N$ of its entries are nonzero.

Spark is a commitment scheme for such sparse multilinear polynomials. Intuitively, the prover commits to the sparse vector u—the nonzero coefficients of the polynomial—and can later prove correct evaluations of its multilinear extension $\tilde{u}$.

Evaluations as inner products.

For a point $r \in \mathbb{F}^{\log N}$, define the dense vector

$$t(r) = \big(L_z(r)\big)_{z \in \{0,1\}^{\log N}}.$$

Then,

$$\tilde{u}(r) = \sum_{z \in \{0,1\}^{\log N}} u(z)\, L_z(r) = \langle u, t(r) \rangle.$$

Evaluating $\tilde{u}$ at r is therefore equivalent to computing the inner product between the sparse vector u and the dense vector $t(r)$. Spark's evaluation proof can thus be viewed as a proof of knowledge of a sparse vector u whose inner product with $t(r)$ equals the claimed value $\tilde{u}(r)$.

From Spark to Surge.

This inner-product perspective generalizes naturally. Instead of restricting attention to $t(r)$—the dense vector determined by Lagrange evaluations—one may consider any *structured* dense vector $T \in \mathbb{F}^N$.

A vector (or table) is said to be *structured* if its multilinear extension $\tilde{T}$ can be evaluated efficiently by a small arithmetic circuit.

The *Surge* protocol extends Spark to this broader setting. It allows the prover to demonstrate knowledge of a sparse vector u and to prove claims of the form

$$\langle u, T \rangle = y,$$

for some efficiently describable vector T and claimed value y. In this way, Surge generalizes Spark's fixed evaluation structure into a flexible proof system for structured inner products.

From Surge to Lasso.

This viewpoint leads directly to Lasso's lookup argument. Suppose the verifier has commitments to a table $t \in \mathbb{F}^n$ and to another vector $a \in \mathbb{F}^m$. The prover wishes to prove that every entry of a appears in t.

This property is captured by the existence of a sparse matrix $M \in \mathbb{F}^{m \times n}$ such that:

1. each row of M contains exactly one entry equal to 1 and all others equal to 0; and
2. $M \cdot t = a$.

Each row of M therefore encodes the position in t of the corresponding entry of a.

Let $\widetilde{M} : \mathbb{F}^{\log m} \times \mathbb{F}^{\log n} \to \mathbb{F}$, $\tilde{a} : \mathbb{F}^{\log m} \to \mathbb{F}$, and $\tilde{t} : \mathbb{F}^{\log n} \to \mathbb{F}$ be the multilinear extensions of M, a, and t, respectively. Then, for a random $r \in \mathbb{F}^{\log m}$ chosen by the verifier, the equality $M \cdot t = a$ is (up to negligible soundness error) equivalent to the scalar equation:

$$\sum_{y \in \{0,1\}^{\log n}} \widetilde{M}(r, y)\, \tilde{t}(y) = \tilde{a}(r).$$

The left-hand side can be viewed as the inner product between the sparse polynomial $\widetilde{M}(r, \cdot)$ and the dense vector $\tilde{t}$.

Lasso proves this equation directly. The prover commits to the sparse polynomial $\widetilde{M}$ using Spark and employs Surge to verify the structured inner-product equality above. This yields a lookup argument that is highly efficient when the table t is *decomposable*—that is, when $\tilde{t}$ can be expressed as a sum of component polynomials each depending on a small number of variables—or when t is small or otherwise structured.

10.2 A Bird's-Eye View of the Spark Protocol

We now describe Spark's operation from a high level. The prover commits to a sparse multilinear polynomial

$$g : \{0,1\}^{\log N} \to \mathbb{F},$$

specified by its nonzero entries at $m \ll N$ locations in the Boolean hypercube. The goal is to enable the prover to later convince the verifier of a claimed evaluation

$$g(r) = v, \qquad r \in \mathbb{F}^{\log N},$$

with costs that depend only on the sparsity m, rather than the full domain size $N = 2^{\log N}$.

Naïve evaluation.

A direct evaluation of $g(r)$ would iterate over all nonzero $z \in \{0,1\}^{\log N}$ with $g(z) \neq 0$, compute $L_z(r)$, and sum $g(z)L_z(r)$. Since each L_z depends on $\log N$ variables, computing it requires $O(\log N)$ time per term, for total cost $O(m \log N)$—better than $O(N)$ but still suboptimal for large $\log N$.

Time-optimal evaluation via variable decomposition.

Spark improves this to total cost $O(c \cdot m)$ for a parameter c such that

$$\log N = c \cdot \log m \quad (\text{i.e., } N = m^c).$$

We decompose the evaluation point r and the nonzero locations z into c blocks:

$$r = (r_1, \ldots, r_c), \quad r_i \in \mathbb{F}^{\log m}, \qquad z = (z_1, \ldots, z_c), \quad z_i \in \{0,1\}^{\log m}.$$

By the product structure of the multilinear Lagrange basis,

$$L_z(r) = \prod_{i=1}^{c} L_{z_i}(r_i).$$

This yields a time-optimal two-phase algorithm:

(1) For each block r_i, evaluate all m possible $L_{z_i}(r_i)$ values in $O(m)$ time, and store them in c read-only memories.

(2) For each nonzero term $z = (z_1, \ldots, z_c)$, read $L_{z_i}(r_i)$ from each memory, multiply the c retrieved values, multiply by $g(z)$, and accumulate the sum.

Each sparse term thus costs $O(c)$ operations, for total runtime $O(c \cdot m)$.

Proving correct execution.

In a proof system, the prover must not only output the claimed value $v = g(r)$ but also convince the verifier that the computation followed this algorithm faithfully, using only the committed sparse g.

Spark models the evaluation as a sequence of memory reads into c tables, each holding precomputed values $\{L_{z_i}(r_i)\}_{z_i}$. The prover then multiplies the retrieved entries and accumulates results according to the sparse support of g.

To make this verifiable, Spark uses an *offline memory checking* (OMC) protocol. The OMC enforces that all reads and writes are consistent with a valid execution trace and that the final accumulated value matches the claimed $g(r)$. This check is implemented using a randomized *grand-product fingerprint*, which guarantees—except with negligible probability—that the prover's memory trace corresponds to a correct execution. We define this fingerprinting technique formally in the next chapter.

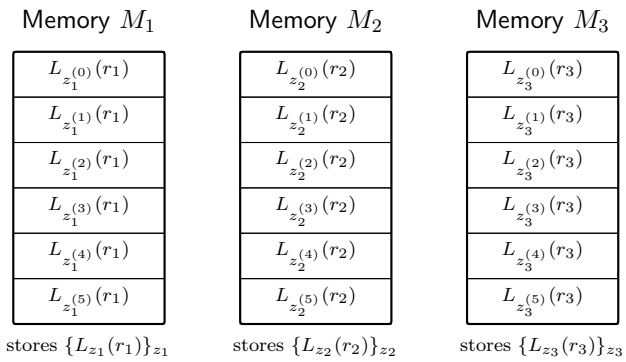

Figure 10.1: Example with $c = 3$ and $m = 6$. Each memory block M_i stores all evaluations $\{L_{z_i}(r_i)\}_{z_i}$ of the $\log m$-variate Lagrange basis polynomials. For each nonzero term $z = (z_1, \ldots, z_c)$, the prover looks up $L_{z_i}(r_i)$ from each M_i, multiplies across blocks, multiplies by $g(z)$, and accumulates. Offline memory checking enforces consistency of all reads.

10.2.1 The $c = 2$ Case: A Simpler Transformation

To build intuition for Spark's general construction, we begin by analyzing the case $c = 2$, where the input variables are split into two equal parts and the evaluation algorithm uses

two memory blocks. This restricted setting captures all essential ideas of the protocol while keeping notation simple; we later show how the same construction extends naturally to arbitrary $c \geq 2$. This corresponds to splitting the input variables into two equal parts, and thus using only $c = 2$ memory blocks in the evaluation algorithm.

Let

$$D : \mathbb{F}^{2 \log M} \to \mathbb{F}$$

be the multilinear extension of a sparse function that is nonzero at at most m points in $\{0,1\}^{2 \log M}$. Our goal is to express D in a form that allows efficient evaluation using two memory blocks, and efficient commitment using $(\log m)$-variate dense polynomial commitments.

Representing Sparse Polynomials by Dense Ones. Suppose D is nonzero at at most m locations of $\{0,1\}^{2 \log M}$. Each nonzero point can be written as a triple $(i, j, D(i,j))$, where $i, j \in \{0,1\}^{\log M}$ and $D(i,j) \neq 0$.

We collect the m nonzero entries of D into three vectors:

$$R[k] = \texttt{to-field}(i_k), \qquad C[k] = \texttt{to-field}(j_k), \qquad V[k] = D(i_k, j_k),$$

for each $k \in \{0, \ldots, m-1\}$.

We then define three multilinear polynomials

$$\mathsf{row}, \mathsf{col}, \mathsf{val} : \mathbb{F}^{\log m} \to \mathbb{F}$$

as the unique multilinear extensions satisfying, for all $k \in \{0,1\}^{\log m}$,

$$\mathsf{row}(k) = R[k], \qquad \mathsf{col}(k) = C[k], \qquad \mathsf{val}(k) = V[k].$$

Rewriting the Evaluation. Let $r = (r_x, r_y) \in \mathbb{F}^{2 \log M}$ be the point where we wish to evaluate D. We can now express this evaluation as:

$$D(r_x, r_y) = \sum_{k \in \{0,1\}^{\log m}} \mathsf{val}(k) \cdot \widetilde{\mathrm{eq}}(\texttt{to-bits}(\mathsf{row}(k)), r_x) \cdot \widetilde{\mathrm{eq}}(\texttt{to-bits}(\mathsf{col}(k)), r_y). \quad (10.1)$$

Derivation of the Sparse Evaluation Formula. The expression in Equation (10.1) follows from the standard decomposition of a multilinear polynomial in the Boolean Lagrange basis.

Since $D : \mathbb{F}^{2\log M} \to \mathbb{F}$ is the multilinear extension of a function supported on at most m points of $\{0,1\}^{2\log M}$, it admits the representation

$$D(r_x, r_y) = \sum_{\substack{(i,j)\in\{0,1\}^{\log M}\times\{0,1\}^{\log M} \\ D(i,j)\neq 0}} D(i,j)\,\widetilde{\mathrm{eq}}(i, r_x)\,\widetilde{\mathrm{eq}}(j, r_y),$$

where $\widetilde{\mathrm{eq}}(\cdot,\cdot)$ denotes the multilinear extension of the Boolean equality predicate. By construction, the nonzero values of D are indexed by $k \in \{0,1\}^{\log m}$, such that each pair (i,j) with $D(i,j) \neq 0$ is represented uniquely by the triple

$$\mathsf{row}(k) = \texttt{to-field}(i), \qquad \mathsf{col}(k) = \texttt{to-field}(j), \qquad \mathsf{val}(k) = D(i,j).$$

Substituting this reindexing into the expansion above yields Equation (10.1), which expresses the evaluation of D at (r_x, r_y) as a sum over the encoded nonzero entries using the $(\log m)$-variate polynomials row, col, and val. This formulation isolates the dependence on (r_x, r_y) into independent basis evaluations, enabling verification via structured memory access in the evaluation protocol.

10.2.2 Evaluation Phase

Having committed to the sparse polynomial D via the three $(\log m)$-variate multilinear polynomials row, col, and val, the next step is to prove that an evaluation $D(r_x, r_y) = v$ is correct for a given $(r_x, r_y) \in \mathbb{F}^{\log M} \times \mathbb{F}^{\log M}$ and claimed value $v \in \mathbb{F}$.

Simulating Memory Reads. The algorithm described above evaluates D by computing:

$$\widetilde{\mathrm{eq}}(\texttt{to-bits}(\mathsf{row}(k)), r_x) \quad \text{and} \quad \widetilde{\mathrm{eq}}(\texttt{to-bits}(\mathsf{col}(k)), r_y)$$

for each $k \in \{0,1\}^{\log m}$, multiplying the result with $\mathsf{val}(k)$, and summing.

To simulate this in the proof system, the prover provides two additional $(\log m)$-variate

multilinear oracles:

$$\mathsf{Erx}(k) = \widetilde{\mathrm{eq}}(\texttt{to-bits}(\mathsf{row}(k)), r_x), \qquad \mathsf{Ery}(k) = \widetilde{\mathrm{eq}}(\texttt{to-bits}(\mathsf{col}(k)), r_y).$$

These two oracles represent the evaluations that the algorithm would read from memory during execution. If the prover is honest, then the equality

$$v = \sum_{k \in \{0,1\}^{\log m}} \mathsf{val}(k) \cdot \mathsf{Erx}(k) \cdot \mathsf{Ery}(k)$$

holds through a standard sum-check protocol.

Lack of Soundness. However, this first attempt is not yet sound. A cheating prover could choose arbitrary oracles Erx and Ery that satisfy the above equation without corresponding to the actual basis evaluations. To guarantee correctness, the verifier must therefore check that for all $k \in \{0,1\}^{\log m}$:

$$\mathsf{Erx}(k) = \widetilde{\mathrm{eq}}(\texttt{to-bits}(\mathsf{row}(k)), r_x), \qquad \mathsf{Ery}(k) = \widetilde{\mathrm{eq}}(\texttt{to-bits}(\mathsf{col}(k)), r_y),$$

which is enforced through the offline memory checking protocol.

10.2.3 Offline Memory Checking

To enforce that the prover's claimed memory reads correspond to valid lookups, Spark employs a technique known as *offline memory checking* (OMC) which was originally introduced by Blum et al. [Blu+91] as a method for verifying the correctness of read operations against an untrusted memory.

This idea originates from classical work in program verification: a trusted checker observes a sequence of read operations issued to an untrusted memory and verifies that each read returns the value most recently written to the corresponding address.

For our setting, we consider a simplified variant of this model in which the memory is initialized with known values at every address, and only *read* operations occur thereafter. To facilitate verification, each memory cell maintains a local counter indicating how many times that address has been accessed. Unlike schemes that rely on a global counter, Spark increments these counters locally, eliminating global state and simplifying the checker's

consistency conditions.

Memory Checker State. The checker maintains two multisets representing the evolution of memory over time:

- RS (the Read Set), initially empty;
- WS (the Write Set), initialized with one entry per memory cell: for each $i \in [M]$, include $(i, v_i, 0)$ in WS, where v_i is the value initially stored at address i and the counter is set to 0.

Read Operation. When the memory is queried at address a, it returns a tuple (v, t) representing the value currently stored and its access counter. The checker performs the following sequence of actions:

1. Records the read by adding (a, v, t) to RS;
2. Instructs the memory to locally increment its counter, writing back $(a, v, t+1)$ and appending this new tuple to WS.

Intuition. The counter serves as a lightweight temporal witness of the order of memory accesses. Each cell maintains its own counter that increases monotonically with every read, ensuring that two reads from the same address are always distinguishable by their timestamps. This mechanism prevents two kinds of cheating behavior:

- *Stale reads:* a dishonest prover cannot claim to have read a value that was valid at an earlier time, since such a read would repeat an already used counter value.
- *Fabricated reads:* the prover cannot invent a tuple (a, v, t) that was never written to memory, because each legitimate read must correspond to a unique sequence of local counter increments.

Hence, the checker only needs to verify that every read tuple recorded in RS also appears as the most recent write in WS. This requirement is captured formally by the invariant below.

Correctness Invariant. At the end of all reads, the checker verifies that the memory's responses were consistent with prior writes. Formally:

Theorem 10.1 (Memory Consistency Invariant [Blu+91])**.** *Let $\mathbb{F}$ be a prime field, and let m denote the total number of read operations, with $m < |\mathbb{F}|$. Let RS and WS be the multisets defined above after the memory execution. Then exactly one of the following holds:*

- *If every read operation returns the value that was last written to its address, there exists a multiset S of size M such that*

$$\mathsf{WS} = \mathsf{RS} \cup S.$$

  Intuitively, S represents the final contents of memory after all reads.

- *If any read returns a value different from the most recently written one, then no such multiset S exists.*

Proof. Suppose first that all reads are correct. Each address a starts with an initial tuple $(a, v_a, 0)$ in WS. When the memory is read at time t, it returns the last written value (a, v, t) and immediately appends $(a, v, t+1)$ to WS. Thus, after m reads, WS consists of the initialization tuples together with one new write per read, each corresponding exactly to the incremented counter value. If we let S denote the set of final tuples (a, v, t_{final}) representing the memory state after all reads, we have

$$\mathsf{WS} = \mathsf{RS} \cup S.$$

Conversely, suppose there exists at least one incorrect read returning a value not equal to the most recent write. Let (a, v, t) be the first such tuple added to RS. Because counters are strictly increasing at each address, the tuple (a, v, t) could only have appeared in WS if the memory had previously written it in response to an earlier read. But since this is the first incorrect read, no such prior write exists, and thus (a, v, t) is missing from WS. Consequently, there can be no multiset S satisfying $\mathsf{WS} = \mathsf{RS} \cup S$. This contradiction implies that $\mathsf{RS} \subseteq \mathsf{WS}$ only if all reads were consistent with the memory's true state. □

10.2.4 Counter Polynomials and the Final Evaluation Argument

The offline memory checking logic is implemented algebraically through a family of auxiliary polynomials called *counter polynomials*. These encode the timestamps (or local access counters) associated with memory reads and writes that occur during the sparse evaluation procedure.

Counter Polynomials. Let $M = 2^{\log M}$ denote the memory size, and suppose D has support on m nonzero entries. Each of the m memory accesses made during the evaluation is associated with counter values that reflect how many times the corresponding memory address has been read.

For each memory block, we define:

- $\mathsf{read_tsrow}, \mathsf{read_tscol} \in \mathbb{F}^m$ — the counters returned during each read operation in the row and col memories, assuming honest execution;
- $\mathsf{final_ctsrow}, \mathsf{final_ctscol} \in \mathbb{F}^M$ — the final counter values stored in each memory cell after all reads.

Each of these vectors is then encoded as a multilinear polynomial:

$$\mathsf{read_tsrow},\ \mathsf{read_tscol} : \mathbb{F}^{\log m} \to \mathbb{F},$$
$$\mathsf{final_ctsrow},\ \mathsf{final_ctscol} : \mathbb{F}^{\log M} \to \mathbb{F}.$$

Enforcing the Invariant. Fix $r_x \in \mathbb{F}^{\log M}$. The OMC invariant from the previous subsection must hold for all accesses into the row memory block. Formally, we define:

$$\begin{aligned}
\mathsf{WS} &= \{(\texttt{to-field}(i), \widetilde{\mathrm{eq}}(i, r_x), 0) : i \in \{0,1\}^{\log M}\} \\
&\quad \cup \{(\mathsf{row}(k), \mathsf{Erx}(k), \mathsf{read_tsrow}(k) + 1) : k \in \{0,1\}^{\log m}\}, \\
\mathsf{RS} &= \{(\mathsf{row}(k), \mathsf{Erx}(k), \mathsf{read_tsrow}(k)) : k \in \{0,1\}^{\log m}\}, \\
\mathsf{S} &= \{(\texttt{to-field}(i), \widetilde{\mathrm{eq}}(i, r_x), \mathsf{final_ctsrow}(i)) : i \in \{0,1\}^{\log M}\}.
\end{aligned}$$

If, for all $k \in \{0,1\}^{\log m}$,

$$\mathsf{Erx}(k) = \widetilde{\mathrm{eq}}(\texttt{to-bits}(\mathsf{row}(k)), r_x),$$

then $\mathsf{WS} = \mathsf{RS} \cup \mathsf{S}$. Otherwise, no such S can exist.

A symmetric condition applies to the col memory block, replacing $(\mathsf{row}, \mathsf{Erx})$ with $(\mathsf{col}, \mathsf{Ery})$ and using their respective counter polynomials.

Chapter 11

Lasso: Lookup Arguments from Spark and Surge

Spark provides a mechanism for verifying that sparse polynomial evaluations are performed honestly, using offline memory checking to enforce consistency across all read operations. Yet lookup arguments require more: they must show that each value in a committed vector arises from a committed table. Bridging this gap is the goal of *Lasso*. Lasso extends Spark in two steps. First, a fingerprint-based soundness test gives a succinct way to verify that all memory accesses match the expected multiset structure. Second, the *Surge* protocol generalizes Spark's machinery to structured inner products, allowing the verifier to certify that every accessed index corresponds to an entry in the lookup table. Together, these components yield the Lasso lookup argument developed in the remainder of this chapter.

Fingerprint-Based Soundness. To check this invariant efficiently, Spark applies a randomized fingerprint equality test over the field $\mathbb{F}$. Given two multisets $A, B \subseteq \mathbb{F}^3$, define:

$$H_{\tau,\gamma}(A) = \prod_{(a,v,t)\in A} \big(a\gamma^2 + v\gamma + t - \tau\big),$$

for random $\tau, \gamma \in \mathbb{F}$. Then:

- If $A = B$, the equality $H_{\tau,\gamma}(A) = H_{\tau,\gamma}(B)$ holds with probability 1;
- If $A \neq B$, the equality holds with probability at most $O(|A| + |B|)/|\mathbb{F}|$.

This fingerprint check gives a succinct probabilistic proof that the read and write traces are consistent with an honest execution of the sparse evaluation algorithm.

Evaluation Protocol. We can now summarize the evaluation proof for the Spark commitment scheme. Given a committed sparse polynomial D and a query point $(r_x, r_y) \in \mathbb{F}^{2\log M}$ with claimed value v, the prover must convince the verifier that $D(r_x, r_y) = v$.

The protocol proceeds as follows:

1. **Setup.** The prover exposes six multilinear polynomial oracles:

$$\mathsf{Erx}, \mathsf{Ery},\ \mathsf{read_tsrow}, \mathsf{read_tscol},\ \mathsf{final_ctsrow}, \mathsf{final_ctscol}.$$

2. **Evaluation Check.** The verifier and prover execute a sum-check to verify:

$$v \stackrel{?}{=} \sum_{k \in \{0,1\}^{\log m}} \mathsf{val}(k) \cdot \mathsf{Erx}(k) \cdot \mathsf{Ery}(k),$$

   which follows from Equation (10.1) under the assumption of honest memory reads.

3. **Memory Consistency.** The verifier samples random $\tau, \gamma \in \mathbb{F}$ and applies the fingerprint check to confirm that:

$$H_{\tau,\gamma}(\mathsf{WS}) = H_{\tau,\gamma}(\mathsf{RS}) \cdot H_{\tau,\gamma}(\mathsf{S}),$$

   for both the row and col memories, ensuring that Erx and Ery correspond to valid reads of the precomputed Lagrange tables.

Generalization to $c > 2$. The case $c = 2$ naturally extends to $c > 2$. Just as $\widetilde{\mathsf{eq}}((i,j),(r_x,r_y))$ factorizes as $\widetilde{\mathsf{eq}}(i, r_x) \cdot \widetilde{\mathsf{eq}}(j, r_y)$, the general case satisfies:

$$\widetilde{\mathsf{eq}}((i_1,\ldots,i_c),(r_1,\ldots,r_c)) = \prod_{\ell=1}^{c} \widetilde{\mathsf{eq}}(i_\ell, r_\ell).$$

Each term in this product corresponds to one memory block, each with its own read and counter polynomials. The proof structure and soundness checks remain identical, and the overhead grows linearly with c, or even sublinearly when using batched polynomial

commitments. This modular design allows Spark to extend efficiently to sparse polynomials defined over high-dimensional domains while preserving succinctness and verifier efficiency.

11.1 Lasso

The *Lasso* protocol extends Spark to support lookup constraints. While Spark provides a way to commit to and evaluate sparse multilinear polynomials, Lasso uses the same machinery to prove that every element of a committed vector lies within a committed table. Conceptually, it transforms a lookup relation into the verification of a structured sparse matrix–vector product.

11.1.1 The road from Spark to Lasso

Recalling Spark on matrices. In Spark, the prover commits to a sparse polynomial

$$D(x, y) = \sum_{k=1}^{m} \mathsf{val}(k) \cdot \widetilde{\mathsf{eq}}(\mathsf{row}(k), x) \cdot \widetilde{\mathsf{eq}}(\mathsf{col}(k), y),$$

where each nonzero term corresponds to a triple $(\mathsf{row}(k), \mathsf{col}(k), \mathsf{val}(k))$.

This can be viewed as a sparse encoding of a large matrix: each pair $(\mathsf{row}(k), \mathsf{col}(k))$ gives the coordinates of a nonzero entry, and $\mathsf{val}(k)$ gives its value. The prover stores the Lagrange basis evaluations $\widetilde{\mathsf{eq}}(\mathsf{row}(k), r_x)$ and $\widetilde{\mathsf{eq}}(\mathsf{col}(k), r_y)$ in two independent memory blocks, one for each variable group, and uses offline memory checking to guarantee the integrity of all reads.

Specializing Spark to lookup. Lasso observes that if each row of this matrix contains exactly one nonzero entry equal to 1, then the matrix acts as a *selector* over a table $t \in \mathbb{F}^N$: for a vector $a \in \mathbb{F}^m$,

$$a = M \cdot t,$$

where each row of M selects one element of t. Thus, proving that every a_i appears in t reduces to showing the existence of such a sparse matrix M.

Simplification. Because all nonzero entries of M are equal to 1, the value polynomial val used in Spark becomes unnecessary and can be omitted. Only the *addresses* of the nonzero entries remain relevant. In Spark these addresses were represented by the pair $(\mathsf{row}, \mathsf{col})$, but in Lasso we need to encode a single index of the table t, which can be large. To handle this efficiently, Lasso decomposes the index into several smaller coordinates.

Dimensional encoding of addresses. Let t have N entries, and fix a small constant c. We write each table index as a c-tuple of smaller coordinates, each in $\{0, \ldots, N^{1/c} - 1\}$. Then, instead of committing to two address polynomials $(\mathsf{row}, \mathsf{col})$, the prover commits to c polynomials:

$$\mathsf{dim}_1, \ldots, \mathsf{dim}_c : \{0,1\}^{\log m} \to \{0, \ldots, N^{1/c} - 1\}.$$

For each nonzero term $k \in \{0,1\}^{\log m}$, the tuple

$$(\mathsf{dim}_1(k), \ldots, \mathsf{dim}_c(k))$$

encodes the location of the 1 in the corresponding row of M. These c polynomials generalize Spark's row and col: they collectively describe the structured support of the sparse polynomial $\tilde{M}$, now spread across c independent memory blocks.

Remark 11.1 (From Spark to Lasso).

Concept	**Spark**	**Lasso**
Address polynomials	$\mathsf{row}, \mathsf{col}$	$\mathsf{dim}_1, \ldots, \mathsf{dim}_c$
Value polynomial	val	omitted (constant 1)
Nonzero entries	arbitrary values	exactly one per row
Number of memories	2(in our example)	c
Memory verification	OMC for row/col	OMC for each dim_i

Verifier's challenge and prover messages. The verifier samples a random point

$$(r, r') \in \mathbb{F}^{\log m} \times \mathbb{F}^{c \cdot \log N^{1/c}}, \quad \text{where } r' = (r'_1, \ldots, r'_c),$$

and sends it to the prover. In response, the prover sends:

- c evaluation polynomials
$$\mathsf{E}_i(k) = \widetilde{\mathsf{eq}}(\mathsf{dim}_i(k), r'_i),$$
one per memory block, each giving the Lagrange basis evaluation at the chosen index;

- c read counter polynomials $\mathsf{read_ts}_i$ and c final counter polynomials $\mathsf{final_cts}_i$, which track and fix the access order of each memory cell, ensuring read integrity through the offline memory checking protocol.

Evaluation equation. The verifier expects the evaluation of $\tilde{M}$ at (r, r') to satisfy

$$\tilde{M}(r, r') = \sum_{k \in \{0,1\}^{\log m}} \widetilde{\mathsf{eq}}(r, k) \cdot \prod_{i=1}^{c} \mathsf{E}_i(k).$$

This is a direct analogue of Spark's two-memory case: the pair $(\mathsf{row}, \mathsf{col})$ has been replaced by $(\mathsf{dim}_1, \ldots, \mathsf{dim}_c)$, and the product of $\mathsf{E}_i(k)$ terms corresponds to multiplying across all c memory lookups.

Intuition. Each nonzero term k represents one row of M:

1. The prover reads from each memory block i the value
$$\mathsf{E}_i(k) = \widetilde{\mathsf{eq}}(\mathsf{dim}_i(k), r'_i),$$
corresponding to the basis evaluation for coordinate $\mathsf{dim}_i(k)$;

2. These values are multiplied across all c blocks to form the full Lagrange basis evaluation;

3. The result is multiplied by the fixed $\mathsf{val}(k) = 1$ and accumulated, weighted by $\widetilde{\mathsf{eq}}(r, k)$.

This mirrors Spark's sparse evaluation, except that Lasso generalizes the number of memories and eliminates the explicit value polynomial.

Soundness checks. To ensure correctness, the verifier performs:

1. A sum-check protocol to verify

$$v \stackrel{?}{=} \sum_{k \in \{0,1\}^{\log m}} \widetilde{\mathsf{eq}}(r, k) \cdot \prod_{i=1}^{c} \mathsf{E}_i(k),$$

   where v is the claimed value of $\tilde{M}(r, r')$.

2. A point check at a random k^* to confirm that

$$\mathsf{E}_i(k^*) = \widetilde{\mathsf{eq}}(\mathsf{dim}_i(k^*), r'_i),$$

   for each memory block i.

3. An offline memory checking protocol (Section 10.2.3) to verify that each E_i was read honestly, using the counters $\mathsf{read_ts}_i$ and $\mathsf{final_cts}_i$.

4. A fingerprint-based consistency check:

$$H_{\tau_i,\gamma_i}(\mathsf{WS}_i) \stackrel{?}{=} H_{\tau_i,\gamma_i}(\mathsf{RS}_i) \cdot H_{\tau_i,\gamma_i}(\mathsf{S}_i),$$

   where for each i:

$$\begin{aligned}
\mathsf{RS}_i &= \{(\mathsf{dim}_i(k), \mathsf{E}_i(k), \mathsf{read_ts}_i(k))\}_k, \\
\mathsf{WS}_i &= \{(\mathsf{dim}_i(k), \mathsf{E}_i(k), \mathsf{read_ts}_i(k) + 1)\}_k, \\
\mathsf{S}_i &= \{(\texttt{to-field}(j), \widetilde{\mathsf{eq}}(j, r'_i), \mathsf{final_cts}_i(j))\}_j.
\end{aligned}$$

11.1.2 Surge: Extending Spark to Structured Inner Products

The *Surge* protocol generalizes Spark to support inner product relations involving structured, dense vectors. While Spark enables commitments to sparse multilinear polynomials and proofs of their evaluations, Surge extends this functionality to verify statements of the form

$$\sum_{j \in \{0,1\}^{\log N}} \tilde{M}(r, j) \cdot \tilde{t}(j),$$

where $r \in \mathbb{F}^{\log m}$ is a random challenge from the verifier, $\tilde{M}$ is the multilinear extension of a sparse matrix M, and $\tilde{t}$ is the multilinear extension of a dense table $t \in \mathbb{F}^N$.

Structured (SOS) tables. To enable efficient proofs, Surge assumes that the lookup table t has a structured form, called *Spark-Only Structure* (SOS). Concretely, there exists a small constant k and a function

$$g : \mathbb{F}^\alpha \to \mathbb{F}, \quad \text{where } \alpha = kc,$$

such that each table entry $t[r]$ can be computed as

$$t[r] = g\big(T_1[r_1], \ldots, T_k[r_1], T_{k+1}[r_2], \ldots, T_{2k}[r_2], \ldots, T_{\alpha-k+1}[r_c], \ldots, T_\alpha[r_c]\big),$$

where $r = (r_1, \ldots, r_c)$ decomposes the index into c coordinates, and each T_i is a dense subtable of size $N^{1/c}$. This decomposition allows each coordinate of the lookup index to be handled independently, in the same way Spark used separate memories for the row and col components.

Inner product expansion. Under this decomposition, the inner product between $\tilde{M}$ and $\tilde{t}$ can be written as

$$\sum_{i \in \{0,1\}^{\log m}} \widetilde{\mathsf{eq}}(i, r) \cdot g\big(T_1[nz_1(i)], \ldots, T_\alpha[nz_c(i)]\big),$$

where each $nz_j(i)$ denotes the j-th coordinate of the unique nonzero entry in the i-th row of M. This expression generalizes Spark's sparse evaluation by replacing direct basis products with a structured combination defined by g.

Protocol overview. To prove this relation, Surge reuses Spark's commitment structure. The prover commits to the same address polynomials $\mathsf{dim}_1, \ldots, \mathsf{dim}_c$, where each dim_j specifies the j-th coordinate of the nonzero entry in the i-th row of M. Then, for each subtable T_i, the prover additionally commits to:

- an evaluation polynomial E_i, defined by

$$\mathsf{E}_i(k) = T_i[\mathsf{dim}_i(k)],$$

- a read counter polynomial $\mathsf{read_ts}_i$, and
- a final counter polynomial $\mathsf{final_cts}_i$.

Sum-check relation. The verifier and prover engage in a sum-check protocol to verify that

$$h(k) = \widetilde{\mathsf{eq}}(r, k) \cdot g\big(\mathsf{E}_1(k), \ldots, \mathsf{E}_\alpha(k)\big)$$

has the claimed evaluation at the random challenge r. At the conclusion of sum-check, the verifier checks that each E_i correctly corresponds to the value read from its subtable by verifying

$$\mathsf{E}_i(k) = T_i[\mathsf{dim}_i(k)].$$

These consistency checks are performed via the same fingerprint-based offline memory checking protocol used in Spark, ensuring that each read operation is consistent with the memory trace defined by the counter polynomials.

11.1.3 The Lasso Lookup Argument

Finally, Lasso combines the Surge inner product protocol with Spark's sparse commitment scheme to realize an efficient lookup argument. It allows a prover to demonstrate that each element of a committed vector $a \in \mathbb{F}^m$ appears in a structured table $t \in \mathbb{F}^N$, without revealing which entries were accessed.

Input.

- A commitment to the multilinear extension $\tilde{a}$ of a vector $a \in \mathbb{F}^m$;
- A structured table $T : \{0,1\}^{\log N} \to \mathbb{F}$ with known SOS decomposition.

Protocol steps.

1. **Commitment.**
   The prover commits to the multilinear extension $\tilde{M}$ of an $m \times N$ matrix $M \in \{0,1\}^{m \times N}$ where each row is a unit vector. This commitment consists of c polynomials:

$$\mathsf{dim}_1, \ldots, \mathsf{dim}_c : \{0,1\}^{\log m} \to \{0, \ldots, N^{1/c} - 1\},$$

specifying the position of the 1 in each row, decomposed into c coordinates.

2. **Verifier challenge.**

   The verifier samples a random $r \in \mathbb{F}^{\log m}$ and sends it to the prover.

3. **Query phase.**

   The verifier queries $\tilde{a}$ at r, obtaining the claimed value $\tilde{a}(r)$.

4. **Evaluation proof via Surge.**

   The prover and verifier invoke the SURGE protocol to verify that

$$\sum_{y \in \{0,1\}^{\log N}} \tilde{M}(r, y) \cdot \tilde{t}(y) = \tilde{a}(r),$$

   where $\tilde{t}$ is the multilinear extension of t computed via its SOS decomposition. The sum-check protocol ensures the correctness of the evaluation, and the fingerprint-based memory checks guarantee that each E_i indeed represents a consistent read from its respective subtable.

Soundness and efficiency. If the prover attempts to cheat by providing inconsistent evaluations—either by reading from incorrect memory addresses or by manipulating counter values—the offline memory checking protocol detects the mismatch with overwhelming probability. The use of c memory blocks, one per dimension, allows all checks to be performed in parallel, and the total communication and verification overhead grows only linearly in c. In practice, c is a small constant (typically 2 or 3), so the protocol remains succinct and efficient.

11.1.4 Cost Analysis and Comparison

Cost of Spark. Spark achieves the best-known prover efficiency among polynomial commitment schemes that satisfy the standard definition of computational binding. Its core advantage is that the prover's work grows *linearly* in the number of nonzero coefficients of the committed polynomial, rather than in the total size of its domain.

Concretely, suppose we commit to a multilinear polynomial $g : \mathbb{F}^{c \log M} \to \mathbb{F}$ that is m-sparse over a Boolean hypercube of size $N = M^c$. In Spark, the prover commits to:

- $3c + 1$ polynomials over $\log m$ variables (corresponding to the dim, E, and counter polynomials), and
- $2c$ polynomials over $\log M$ variables (corresponding to the memory initialization and final counter states).

Each such commitment is performed via a standard dense multilinear polynomial commitment scheme (e.g., Spartan's polynomial IOP), whose cost is linear in the number of coefficients. Hence, the total prover cost is

$$\mathcal{O}(m) + \mathcal{O}(c \cdot M),$$

dominated by the number of nonzero coefficients m when $m \ll N$.

Verifier work, in contrast, is only polylogarithmic in m and M: the verifier performs a small number of oracle queries and sum-check verifications, with communication complexity $\mathcal{O}(\log m + \log M)$. Thus, Spark achieves **succinct verification with sparse proving cost**. This contrasts with classical polynomial commitments such as KZG or DARK, where prover time scales linearly in N, the total number of coefficients.

Comparison with other polynomial commitment schemes. In standard polynomial commitment schemes, the prover's work scales with the full degree of the polynomial, even when most coefficients are zero. Spark instead operates directly on a sparse representation, evaluating the polynomial via structured memory lookups of precomputed Lagrange basis values. Offline memory checking ensures that all such reads are consistent with a valid execution trace. As a result, the prover's time scales linearly in the number of nonzero terms.

Cost of Lasso. Lasso inherits Spark's $\mathcal{O}(m)$ prover cost while adding a modest overhead to handle the lookup structure. Each lookup proof introduces:

- c address polynomials $\mathsf{dim}_1, \ldots, \mathsf{dim}_c$,
- c evaluation polynomials $\mathsf{E}_1, \ldots, \mathsf{E}_c$,
- $2c$ counter polynomials ($\mathsf{read_ts}_i$, $\mathsf{final_cts}_i$ for each i).

All of these are multilinear polynomials over $\log m$ or $\log M$ variables, each committed via the same dense polynomial commitment primitive used in Spark. Thus, the asymptotic complexity remains linear in the number of lookups m:

$$\text{Prover time: } \mathcal{O}(m), \quad \text{Verifier time: } \mathcal{O}(\log m), \quad \text{Proof size: } \mathcal{O}(\log m).$$

The additional cost of Lasso arises only from verifying the structured inner product

$$\sum_{y \in \{0,1\}^{\log N}} \tilde{M}(r, y) \cdot \tilde{t}(y) = \tilde{a}(r),$$

which is handled by the Surge protocol using the same sum-check and memory checking primitives as Spark. The only extra work stems from evaluating the structured function g that defines the SOS decomposition of the lookup table.

Comparison with other lookup arguments. Lasso's lookup mechanism departs significantly from prior approaches such as Plookup, Logup, and Logup combined with GKR. Plookup enforces inclusion by comparing multiplicative grand products across the lookup and witness domains, which requires maintaining auxiliary columns and large cross-product computations that scale with the table size N. Logup replaces this multiplicative structure with an additive one based on logarithmic derivatives, removing the grand products but still requiring several helper polynomials to express rational inverses. The Logup–GKR construction further improves this by verifying the logarithmic derivative identity through fractional sumchecks within a GKR circuit, eliminating all helper commitments and reducing verification to a linear computation over the Boolean hypercube.

Lasso takes a different route. Instead of expressing lookups through multiplicative or rational equalities, it verifies table membership through sparse inner products and multiset fingerprint checks. The cost of this procedure depends only on the number of lookups m rather than the total table size N. In addition, Lasso supports structured lookup tables defined by the Spark-Only Structure (SOS) property, allowing small dense tables to be reused across multiple lookups. This design yields asymptotic improvements when the same table is accessed repeatedly, while remaining fully compatible with the polynomial IOP framework used by modern SNARKs.

Chapter 12

Jolt: Lookup-Based RISC-V Emulation

12.1 Introduction and Motivation

The development of Jolt [Tha+24] is a natural continuation of the ideas introduced with Lasso. In the previous chapter, lookup arguments were shown to provide efficient proofs of membership in structured tables by combining multilinear extensions with a compact grand-product check. Jolt builds directly on this foundation: its goal is to emulate the execution of a RISC-V processor while expressing all instruction semantics purely as algebraic lookups rather than circuit constraints.

This direction follows the vision known as the *lookup singularity*, introduced by Setty, Thaler, and Wahby [STW23]. The underlying idea is that the semantics of any computation can be encoded as lookups into well-structured tables, eliminating circuit-specific arithmetic reasoning. Once the lookup mechanism is sufficiently expressive, it captures the full behavior of a virtual machine with minimal constraint overhead. The verifier then needs only to check that lookup queries correspond to valid table entries and that the committed execution trace remains consistent over time.

Jolt realizes this principle concretely by combining three algebraic components:

1. A minimal formal model of the RISC-V architecture that defines the state transition to be verified.

2. A library of multilinear and decomposable lookup tables, each representing the semantics of an instruction class.

3. A composition layer that commits to the multilinear extension of the execution trace

and uses Lasso to certify that every step corresponds to a valid lookup.

12.2 Minimal Machine Model and Problem Statement

Jolt targets a subset of the RISC-V instruction set architecture that is expressive enough to capture real computation while remaining algebraically simple. A program is viewed as a deterministic state machine over the tuple

$$(\mathrm{PC}, R, M),$$

where PC is the program counter, $R = (r_0, r_1, \ldots, r_{31})$ is the register file, and M is a byte-addressable memory array. Each instruction defines a transition between two such states that is entirely determined by the current state and the instruction word.

An instruction is represented as a 5-tuple

$$[\texttt{opcode}, rs_1, rs_2, rd, \texttt{imm}],$$

where `opcode` selects the operation, rs_1 and rs_2 identify the source registers, rd is the destination register, and `imm` is an immediate value encoded within the instruction. The instruction semantics specify how the next state (PC', R', M') is computed from (PC, R, M).

The goal of the proof system is to verify that a sequence of such transitions forms a valid execution trace of the RISC-V machine for a given program and input. Formally, let z_t denote the encoded state of the virtual machine at step t, and let step denote the transition relation induced by the instruction at that step. Correct execution requires that

$$z_{t+1} = \mathsf{step}(z_t)$$

for all t in the trace. Jolt expresses this relation through algebraic lookup constraints rather than Boolean circuit constraints, ensuring that each state transition corresponds to a valid instruction semantics.

The memory M is modeled as a byte-addressable array whose consistency is verified through an offline memory-checking argument, as in Spark. The correctness of register and control flow updates, as well as the handling of immediates, are all enforced locally within the lookup-based CPU step circuit.

12.3 Lookup Design

Every instruction in Jolt is verified through a single lookup relation that captures its full semantics. At each CPU step, the prover constructs a tuple of operands derived from the current machine state and performs a lookup into the corresponding evaluation table. Formally, each step enforces a constraint of the form

$$T[\texttt{opcode} \,\|\, z] = r,$$

where z encodes the operands and immediate fields, and r is the result value to be written into the destination register. This equation represents the unified interface between the CPU state and the lookup library.

Since different opcodes correspond to distinct operation semantics, Jolt introduces a *collation polynomial* that routes lookups to the correct evaluation table based on the opcode bits. Let $w = (w_1, \ldots, w_8)$ denote the binary representation of the opcode, and let $g_i(x)$ denote the *operation polynomial* that captures the semantics of operation i. The global collation polynomial is defined as

$$g(w, x) = \sum_{y \in \{0,1\}^8} \mathsf{EQ}_f(w, y) \cdot g_{\mathrm{int}(y)}(x),$$

which ensures that $g(\mathrm{bin}(i), x) = g_i(x)$ for each opcode i. This construction allows a single unified lookup framework to handle all instruction classes while preserving opcode-specific semantics.

This routing mechanism allows all operations to share the same algebraic interface and enables batched verification of multiple opcode classes under Lasso.

To illustrate this pattern, consider the equality test between two W-bit operands. The corresponding table entry is determined by the multilinear function

$$\mathsf{EQ}_f(x, y) = 1 - \sum_{i=0}^{W-1} (x_i - y_i)^2,$$

which evaluates to one if and only if $x = y$. The lookup relation $T[\texttt{opcode} \,\|\, x, y] = \mathsf{EQ}_f(x, y)$ therefore enforces equality as a single table inclusion rather than as a constraint system of size W.

Each operation type reuses this same template with its own evaluation table and operation polynomial, producing a uniform algebraic layer that is directly compatible with the Lasso lookup argument.

12.4 Decomposability Library

The efficiency of Jolt depends on representing every operation table as a *multilinear extension (MLE)* that can be decomposed into smaller subtables. This property allows the lookup argument to operate with logarithmic verification cost in the table size while keeping prover commitments compact. Each RISC-V instruction in Jolt is assigned such an extension polynomial. Logical and arithmetic operations correspond to low-degree multilinear functions, and their evaluation cost for the verifier remains logarithmic in the input dimension.

12.4.1 Core Building Blocks

Three primitive polynomials form the foundation of Jolt's lookup library. They capture equality, comparison, and bit shifting at the level of multilinear extensions.

Equality.

$$\mathsf{EQ}_f(x, y) = 1 - \sum_{i=0}^{W-1} (x_i - y_i)^2.$$

Unsigned comparison.

$$\mathsf{LTU}_g(x, y) = \sum_{i=0}^{W-1} 2^i (1 - y_i) \prod_{j>i} \mathsf{EQ}_f(x_j, y_j).$$

Shift-left logic.

$$\mathsf{SLL}_h(x, s) = \sum_{i=0}^{W-1} 2^i \, x_{i-s},$$

where indices are interpreted modulo W and bits shifted out of range are set to zero.

These functions are multilinear and decomposable and can be evaluated efficiently. Together they serve as the core primitives used to construct all higher-level operation tables in Jolt.

12.4.2 Instruction Composition

Arithmetic and logical instructions are composed by combining these primitives. For example, the `ADD` and `SUB` tables incorporate carry propagation as an auxiliary lookup; comparison and branch instructions reuse EQ_f and LTU_g; shift and rotation operations build on SLL_h. Because each of these components admits a low-degree multilinear form, the resulting tables remain compact and compatible with Lasso's decomposability requirement.

This structure forms the reusable library underlying Jolt's instruction semantics and ensures that all opcode classes can be verified through the same algebraic mechanism.

12.5 Architecture and Composition with Lasso

The Jolt proof system combines the multilinear execution trace, decomposable lookup tables, and the Lasso argument into a unified zkSNARK architecture. This section outlines how these ingredients compose algebraically to verify RISC-V execution succinctly.

12.5.1 Execution Trace Commitment

Let z_t denote the encoded machine state at step t and $\widetilde{z}$ its multilinear extension over the Boolean hypercube $\{0,1\}^n$. The prover commits to $\widetilde{z}$ using a polynomial commitment scheme such as Spartan or Brakedown, which supports low-degree evaluation proofs in time polylogarithmic in the trace length. This commitment binds the prover to the entire execution trace while allowing efficient spot checks during verification.

The constraint system describing valid transitions is written in R1CS form

$$Az \circ Bz = Cz,$$

where each row enforces the local consistency of registers, program counter, and memory. The matrices A, B, C are fixed and sparse, so their evaluations can be computed with logarithmic effort using multilinear evaluation oracles.

12.5.2 Lookup Verification with Lasso

Each instruction step enforces that the relevant operands from $\widetilde{z}$ belong to a decomposable instruction table. For a table split into c subtables, Lasso performs c parallel lookups and verifies multiset equality between queried values and table entries via its grand product argument:

$$\prod_{i=1}^{m}(1+\alpha t_i) = \prod_{i=1}^{m}(1+\alpha s_i),$$

where (t_i) are committed lookups and (s_i) are elements of the reference table. This ensures that every lookup issued by the prover corresponds to a valid instruction semantics entry without revealing intermediate values.

Routing between subtables is handled by the opcode bits and the collation polynomial $g(w, x)$ introduced earlier, ensuring that each step's lookup query is directed to the correct subtable. The verifier runs a single Lasso instance per instruction class, each of logarithmic complexity in its table size.

12.5.3 Virtual Instructions

Complex opcodes such as multiplication and division are implemented as short sequences of virtual instructions executed over an extended register set. Intermediate results are stored in virtual registers that do not modify the architectural state until the final step of the sequence. This technique allows the system to emulate multi-step arithmetic operations using simple MLE lookups without expanding the table size exponentially.

For example, a signed multiplication sequence uses intermediate moves and unsigned multiplications followed by addition of sign-adjusted partial products. Division and remainder operations use non-deterministic `ADVICE` instructions to introduce quotient and remainder values that are then checked through equality and less-than lookups.

12.5.4 Memory Checking

Memory consistency follows the same model as in Spark. Each load or store instruction commits to byte values and timestamps, verifies that timestamps are ordered, and uses small lookups to enforce range checks and less-than relations. This logic is modular and treated as an independent subprotocol, so the CPU core can remain purely lookup-based.

Through this composition, Jolt achieves a uniform algebraic structure in which every step of program execution reduces to a combination of polynomial commitments and lookup verifications under Lasso.

12.6 Costs and Takeaways

The cost of proving RISC-V execution in Jolt depends primarily on the number and bit-length of the elements committed at each CPU step. The analysis presented in this section follows the soundness and complexity bounds established in the original Jolt construction [Tha+24].

Since every instruction is expressed through lookup tables and polynomial commitments, the dominant cost arises from commitment operations rather than arithmetic constraint evaluation.

12.6.1 Per-Step Elements

Each CPU step includes several small elements: about twenty single-bit flags and control signals, three five-bit register indices, and logarithmic-size program and step counters. In addition, each step involves up to three W-bit operands or results, an optional advice value, and the output of a lookup table. Together, these yield approximately fifty elements for 32-bit configurations and sixty-five for 64-bit ones, corresponding to about six commitments to 256-bit field elements under Pippenger's multi-scalar multiplication algorithm.

12.6.2 Memory Operations

Memory instructions follow a slightly different pattern. They rely on offline memory checking, where each accessed byte is paired with a timestamp and verified to appear in the correct order. A load or store of k bytes therefore requires roughly

$$36 + 12k \text{ elements for loads,} \qquad 36 + 13k \text{ elements for stores.}$$

These values include byte flags and range-checked timestamps but exclude any large lookup tables. Because most of these elements have bit-lengths no larger than $\log T$,

the overhead remains nearly constant across program size.

12.6.3 Overall Complexity

The prover's runtime scales linearly with the number of committed elements, dominated by group operations in the commitment phase. The verifier's work is logarithmic in the trace length and constant per instruction class, since each lookup batch under Lasso requires only a small number of field operations. Replacing complex arithmetic circuits by table lookups yields a uniform and auditable proof structure that trades arithmetic depth for commitment simplicity.

12.6.4 Conceptual Outlook

Jolt realizes the lookup singularity proposed by Setty, Thaler, and Wahby by showing that all instruction logic can be captured through decomposable multilinear lookups. This approach eliminates ad hoc gate constructions, producing a zkVM architecture that is modular, efficient, and easy to reason about algebraically. The resulting system connects the theoretical path from sum-check protocols to polynomial commitments and lookup arguments, culminating in a fully algebraic model of computation verification.

Chapter 13

Conclusion

This thesis studied the algebraic structure underlying modern succinct and zero-knowledge proof systems, with a focus on protocols based on multilinear extensions, probabilistic polynomial checks, and lookup arguments. Starting from classical interactive proofs and the sum-check protocol, we developed the tools required to formalize and analyze advanced constructions such as GKR, Spark, Lasso, and Jolt.

A recurring theme throughout the thesis is the reduction of complex verification tasks to low-degree polynomial identities evaluated at random points. Multilinear extensions provide a uniform way to encode discrete computation as algebraic objects, while the Schwartz–Zippel lemma and sum-check protocol supply the core soundness guarantees. Polynomial commitments and lookup arguments build on this foundation, allowing these identities to be enforced succinctly in cryptographic settings.

The later chapters focused on lookup-based proof systems. Spark and Lasso show how sparse computation and table membership constraints can be verified using algebraic memory access patterns together with offline memory checking. Jolt extends these ideas further, demonstrating that the execution of a full RISC-V machine can be verified using lookup relations alone, without reducing instruction semantics to large arithmetic circuits. This shift from circuit constraints to table-based verification reflects a broader trend toward lookup-centric proof system design.

Overall, the constructions studied in this thesis illustrate how a small collection of algebraic techniques can be composed to obtain scalable and expressive proof systems. While many aspects of these protocols are by now well understood, their combination and refinement continue to produce new verification frameworks. The material presented

here is intended to provide a coherent and self-contained account of these developments, serving as a foundation for further work on lookup arguments and algebraic proof systems.

Bibliography

[Bab85] László Babai. "Trading group theory for randomness". In: *Proceedings of the seventeenth annual ACM symposium on Theory of computing*. 1985, pp. 421–429.

[Blu+91] Manuel Blum, W. Evans, Peter Gemmell, Sampath Kannan, and Moni Naor. "Checking the Correctness of Memories". In: *Algorithmica* (1991).

[FS87] Amos Fiat and Adi Shamir. "How to Prove Yourself: Practical Solutions to Identification and Signature Problems". In: *Advances in Cryptology — CRYPTO '86*. Vol. 263. Lecture Notes in Computer Science. Springer, 1987, pp. 186–194. DOI: `10.1007/3-540-47721-7_12`. URL: `https://doi.org/10.1007/3-540-47721-7_12`.

[Gen+13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. "Quadratic Span Programs and Succinct NIZKs without PCPs". In: *Advances in Cryptology – EUROCRYPT 2013*. Vol. 7881. Lecture Notes in Computer Science. Springer, 2013, pp. 626–645.

[GKR15] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. "Delegating Computation: Interactive Proofs for Muggles". In: *Journal of the ACM* 62.4 (2015), pp. 1–64.

[GM84] Shafi Goldwasser and Silvio Micali. "Probabilistic Encryption". In: *Journal of Computer and System Sciences* 28.2 (1984), pp. 270–299.

[GMR85] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. "The Knowledge Complexity of Interactive Proof Systems". In: *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing (STOC '85)*. New York, NY, USA: ACM, 1985, pp. 291–304. DOI: `10.1145/22145.22178`. URL: `https://doi.org/10.1145/22145.22178`.

[GMR89] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. "The Knowledge Complexity of Interactive Proof Systems". In: *SIAM Journal on Computing* 18.1 (1989), pp. 186–208. DOI: 10.1137/0218012. URL: https://doi.org/10.1137/0218012.

[Gro16] Jens Groth. "On the Size of Pairing-Based Non-interactive Arguments". In: *Advances in Cryptology – EUROCRYPT 2016*. Vol. 9665. Lecture Notes in Computer Science. Springer, 2016, pp. 305–326.

[GWC20] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. *Plookup: A Faster Lookup Protocol for Zero-Knowledge Proof Systems*. Cryptology ePrint Archive, Paper 2020/315. 2020. URL: https://eprint.iacr.org/2020/315.

[Hab23] Ulrich Haböck. *LogUp: Lookups via Logarithmic Derivatives and GKR*. Technical note, Orbis Labs (Polygon Zero), March 31, 2023. 2023.

[Lam23] LambdaClass. *Multilinear Polynomials: Survival Kit*. https://blog.lambdaclass.com/multilinear-polynomials-survival-kit/. Accessed: 2025-10-30. 2023.

[Lun+92a] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. "Algebraic Methods for Interactive Proof Systems". In: *Journal of the ACM* 39.4 (1992), pp. 859–868.

[Lun+92b] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. "Algebraic methods for interactive proof systems". In: *Journal of the ACM (JACM)* 39.4 (1992), pp. 859–868.

[Sch80] Jacob T. Schwartz. "Fast Probabilistic Algorithms for Verification of Polynomial Identities". In: *Journal of the ACM* 27.4 (1980), pp. 701–717. DOI: 10.1145/322217.322225. URL: https://doi.org/10.1145/322217.322225.

[Sha92] Adi Shamir. "IP = PSPACE". In: *Journal of the ACM* 39.4 (1992), pp. 869–877.

[STW23] Srinath Setty, Justin Thaler, and Riad S. Wahby. *Unlocking the Lookup Singularity with Lasso*. Cryptology ePrint Archive, Report 2023/1216. Conference version: EUROCRYPT 2024. 2023. URL: https://eprint.iacr.org/2023/1216.

[Tha+24] Justin Thaler, Srinath Setty, Samuel Weisz, and Riad S. Wahby. *Jolt: SNARKs for Virtual Machines via Lookup Arguments*. 2024. arXiv: 2404.06965 [cs.CR].

[Tha22] Justin Thaler. *Proofs, Arguments, and Zero-Knowledge*. Cambridge University Press, 2022. ISBN: 9781108839584. URL: https://www.cambridge.org/9781108839584.

[Zip79] Richard Zippel. "Probabilistic Algorithms for Sparse Polynomials". In: *Proceedings of the International Symposium on Symbolic and Algebraic Computation (EUROSAM '79)*. Vol. 72. Lecture Notes in Computer Science. Springer, 1979, pp. 216–226. DOI: 10.1007/3-540-09519-5_73. URL: https://doi.org/10.1007/3-540-09519-5_73.

[ZKM25] ZKM.IO. *ZKM.IO*. 2025. URL: https://zkm.io (visited on 10/01/2025).